

Laporan Tugas Besar 3

IF2211 Strategi Algoritma

Implementasi Pattern Matching pada Chromium
Browser Extension untuk Deteksi Judi Online

Semester II Tahun 2025/2026



Oleh
NamaKelompok

Anggota:

Wafiq Hibban Robbany (13524016)

Fayyaz Akmal Lauda (13524076)

Syaqina Octavia Rizha (13524088)

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

30 Mei 2026

DAFTAR ISI

DAFTAR ISI.....	1
Bab I.....	2
Deskripsi Tugas.....	2
Bab II.....	4
Landasan Teori.....	4
2.1. Algoritma Knuth-Morris-Pratt.....	4
2.2. Algoritma Boyer-Moore.....	6
2.3. Regular Expression (RegEx).....	8
2.4. Weighted Levenshtein Distance.....	9
2.5. Algoritma Aho-Corasick.....	11
2.6. Algoritma Rabin-Karp.....	12
2.7. Chromium Browser Extension.....	14
Bab III.....	15
Analisis Pemecahan Masalah.....	15
3.1. Analisis Masalah dan Kebutuhan Sistem.....	15
3.2. Strategi Pemrosesan Teks dan Pemetaan Algoritma.....	16
3.3. Arsitektur Extension.....	17
3.4. Contoh Ilustrasi Kasus.....	18
Bab IV.....	21
Implementasi dan Pengujian.....	21
4.1. Implementasi.....	21
4.1.1. Struktur Program.....	21
4.1.2. Implementasi Struktur Data.....	22
4.1.3. Implementasi Program.....	27
4.2. Tata Cara Penggunaan Extension.....	57
4.3. Pengujian.....	58
4.3.1. Hasil Pengujian.....	58
4.3.1.1. Contoh Hasil Exact Matching.....	58
4.3.1.2. Contoh Hasil Exact Matching dan Regex.....	58
4.3.1.3. Contoh Hasil Exact Matching, Regex dan Fuzzy.....	59
4.3.1.4. Contoh Hasil dengan Aho-Corasick dan Rabin-Karp.....	59
4.3.1.5. Contoh Hasil OCR.....	60
4.3.1.6. Contoh Hasil Popup.....	60
4.3.2. Analisis Hasil.....	61
BAB V.....	62
Kesimpulan dan Saran.....	62
5.1. Kesimpulan.....	62
5.2. Saran.....	62
Lampiran.....	63
Repositori Github : Link Github.....	63

Tabel Pembagian Tugas.....	63
Tabel Checklist Fitur.....	63
Pernyataan Generative AI.....	64
Daftar Pustaka.....	65

Bab I

Deskripsi Tugas

Maraknya konten judi online di dunia maya saat ini sering kali memanfaatkan format penamaan yang spesifik untuk menarik perhatian sekaligus menghindari sistem pemblokiran. Fenomena yang biasa kita temukan dalam berbagai situs web saat ini adalah penggunaan format kombinasi `<angka>`, seperti GACOR99, MAXWIN88, maupun MADU308. Tidak hanya itu, para pengembang situs judi online juga kerap menerapkan teknik serupa *phishing* dengan memanfaatkan manipulasi kode komputer atau perbedaan unicode. Mereka menyiasati strategi deteksi dengan menggunakan huruf pengganti yang secara visual mirip bagi manusia, seperti mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi 'l', namun dianggap sebagai karakter yang sepenuhnya berbeda oleh sistem komputer.

Untuk mengatasi permasalahan tersebut, pada Tugas Besar 3 Strategi Algoritma ini, mahasiswa diminta untuk mengembangkan sebuah *Chromium Browser Extension* menggunakan bahasa pemrograman TypeScript yang mampu mendeteksi informasi bermuatan unsur judi online secara cerdas dan otomatis. Ekstensi ini bekerja dengan memindai elemen-elemen di dalam *Document Object Model* (DOM) halaman web secara langsung melalui pendekatan pencocokan string (*string matching*). Sistem dirancang untuk menjalankan pemeriksaan terstruktur mulai dari pencarian kata kunci dasar, analisis pola berbasis angka, hingga perhitungan tingkat kemiripan teks guna memitigasi segala bentuk manipulasi karakter visual.

Komponen-komponen penting dan spesifikasi wajib yang harus diimplementasikan pada tugas besar ini antara lain:

1. Pengembangan Berbasis TypeScript & Exact Matching

Ekstensi ini dikembangkan menggunakan bahasa pemrograman TypeScript. Dalam proses pencocokan kata kunci dasar, sistem wajib mengimplementasikan algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore untuk melakukan pencarian secara iteratif. Proses pencarian ini memanfaatkan berkas bernama `keyword.txt` yang dibaca secara langsung oleh sistem, sehingga berjalan secara otomatis tanpa memerlukan input manual dari pengguna.

2. RegEx Matching untuk Pola Spesifik

Secara paralel atau sekuensial dengan pencarian kata kunci dasar, sistem harus menjalankan *Regular Expression* (RegEx) untuk menangkap pola spesifik `<angka>`. Format pencarian berbasis RegEx ini seminimal mungkin harus mampu menangani variasi 2 atau 3 angka di belakang kata tanpa spasi, seperti MAXWIN234 atau SLOT99, serta dirancang secara detail untuk mengatasi berbagai *edge case* pada dokumen web.

3. Fuzzy Matching dengan Weighted Levenshtein Distance

Apabila suatu string dalam DOM tidak memicu kecocokan sama sekali pada saat *exact matching* namun memiliki indikasi manipulasi teks, sistem akan melakukan pencarian kembali menggunakan

perhitungan tingkat kemiripan *Weighted Levenshtein Distance*. Implementasi algoritma ini memberikan *weighted cost* yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan karakter yang benar-benar berbeda. Nilai *threshold* juga ditentukan secara tepat untuk menjaga relevansi hasil sekaligus menghindari munculnya *false positive*.

4. Hirarki Pengecekan Sistem

Sistem menjalankan proses evaluasi teks dengan urutan hirarki yang jelas. Proses diawali dengan *Exact Matching* untuk mencari kata kunci dasar dari berkas `keyword.txt`. Bersamaan dengan itu, *RegEx Matching* dijalankan untuk menyaring pola kata berangka. Jika string dalam DOM tidak lolos kecocokan persis tetapi terindikasi memiliki kemiripan visual, maka *Fuzzy Matching* akan dijalankan sebagai upaya mendeteksi manipulasi karakter.

5. Manipulasi DOM dan *Highlighting*

Ketika kata kunci judi online berhasil ditemukan pada halaman web, elemen DOM yang terkait harus ditandai dengan sorotan warna bebas. Proses highlight ini diwajibkan untuk tepat sasaran, tidak boleh merusak tata letak layout asli halaman web, serta dapat terhapus secara otomatis ketika sistem melakukan *rescanning*.

6. Tooltip Informasi

Sistem harus menyediakan interaksi dinamis saat pengguna mengarahkan kursor di atas elemen teks halaman yang terdeteksi. Ekstensi akan menampilkan tooltip yang dibuat menggunakan komponen custom DOM element. Tooltip tersebut wajib memuat data berupa kata kunci yang terdeteksi, jenis algoritma pencocokan yang digunakan, jumlah kemunculan kata tersebut, serta catatan waktu eksekusinya.

7. Statistik Real-Time pada Popup

Sebagai pusat informasi bagi pengguna, bagian *popup extension* harus menyediakan visualisasi data statistik pemindaian secara *real-time*. Dashboard statistik ini wajib menampilkan informasi mengenai total seluruh kata kunci yang ditemukan, visualisasi perbandingan jumlah kemunculan antara satu kata kunci dengan yang lain, catatan waktu eksekusi dari tiap-tiap algoritma yang berjalan, serta jumlah total kecocokan, yang berhasil diidentifikasi oleh masing-masing metode.

Bab II

Landasan Teori

2.1. Algoritma Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) merupakan algoritma *string matching* yang dirancang secara spesifik untuk mencari kemunculan sebuah pola di dalam teks utama secara efisien. Kunci utama dari efisiensi algoritma KMP terletak pada kemampuannya dalam meminimalkan operasi perbandingan karakter yang redundan. Algoritma ini menjamin bahwa indeks pembacaan pada teks utama akan selalu bergerak maju dan tidak pernah mengalami proses mundur (*backtrack*). Melalui pendekatan linear ini, pencarian KMP memiliki kompleksitas waktu yang sangat optimal, yaitu $O(n + m)$, di mana n merepresentasikan panjang teks utama dan m merepresentasikan panjang pola pencarian.

Untuk mencapai tingkat efisiensi tersebut, KMP memanfaatkan sebuah struktur data pendukung yang disebut sebagai array LPS (*Longest Proper Prefix which is also Suffix*) atau yang sering dikenal sebagai *Failure Table*. Struktur data pendukung ini memiliki dua karakteristik utama:

- Tabel ini diekstraksi baris demi baris seiring dengan penambahan karakter pada pola pencarian. Setiap elemennya mewakili skenario sejauh mana teks dan pola telah berhasil dicocokkan pada titik tertentu.
- Nilai transisi pada tabel dihitung dengan mengekstraksi panjang maksimum dari *proper prefix* (awalan string) yang secara identik beririsan dengan *proper suffix* (akhiran string) untuk setiap *substring* pola. Nilai inilah yang menjadi peta instruksi mengenai seberapa jauh indeks teks dapat dilompati setelah terjadi ketidakcocokan.

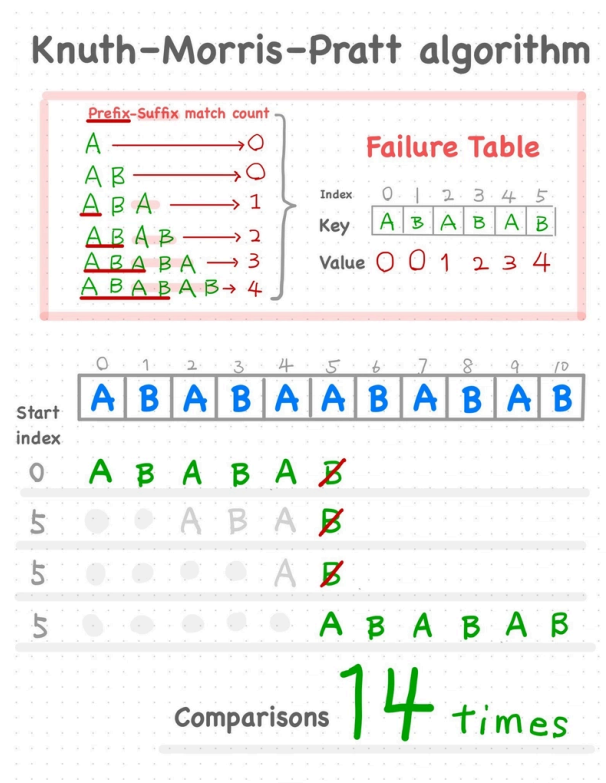
Secara teknis, alur eksekusi algoritma KMP terbagi menjadi dua fase komputasi utama. Fase pertama adalah pembentukan *Failure Table*, dengan langkah-langkah iteratif sebagai berikut:

- Algoritma menginisialisasi dua penunjuk (*pointer*), yaitu penunjuk panjang awalan yang cocok (*prefix length*) dan penunjuk indeks evaluasi karakter pada pola.
- Evaluasi dilakukan sepanjang pola. Jika karakter pada kedua penunjuk sama, algoritma mencatat panjang awalan ke dalam tabel dan memajukan kedua penunjuk.
- Jika terjadi ketidakcocokan dan penunjuk awalan tidak berada di indeks awal, algoritma memundurkan penunjuk awalan merujuk pada nilai tabel sebelumnya, tanpa memajukan penunjuk evaluasi.
- Jika ketidakcocokan terjadi saat penunjuk awalan sudah berada di indeks awal, algoritma mencatat nilai nol pada tabel dan hanya memajukan penunjuk evaluasi karakter.

Setelah *Failure Table* berhasil dibangun, algoritma memasuki fase kedua, yaitu pencarian pola pada teks utama dengan alur evaluasi berikut:

- Algoritma menginisialisasi penunjuk indeks teks dan penunjuk indeks pola pada posisi awal.

- Iterasi dilakukan selama penunjuk indeks teks belum mencapai batas akhir dokumen. Jika karakter pada teks dan pola cocok, kedua penunjuk dimajukan secara bersamaan.
- Jika seluruh karakter pola berhasil dicocokkan, algoritma mengembalikan posisi penemuan tersebut. Penunjuk pola kemudian dimundurkan berdasarkan nilai *Failure Table* untuk melanjutkan pencarian kemungkinan kemunculan berikutnya.
- Namun, apabila terjadi ketidakcocokan di tengah proses perbandingan dan penunjuk pola tidak di indeks awal, penunjuk pola digeser merujuk pada *Failure Table* tanpa memajukan penunjuk teks sama sekali.
- Jika ketidakcocokan terjadi saat penunjuk pola berada di indeks awal, algoritma sekadar memajukan penunjuk teks ke karakter berikutnya.



Gambar 2.1. Ilustrasi Pencocokan KMP dan Mekanisme Pergeseran menggunakan *Failure Table*

<https://dev.to/yo-shi/for-beginners-understanding-the-kmp-algorithm-by-comparing-with-the-brute-force-1da3>

Penerapan alur komputasi di atas dapat dilihat secara konkret melalui ilustrasi pencarian pola ABABAB pada teks ABABAABABAB di Gambar 2.1. Pada percobaan pertama, algoritma berhasil mencocokkan lima karakter, namun menemukan *mismatch* pada karakter keenam. Alih-alih mengulang pembacaan dari awal, mekanisme pergeseran KMP bekerja dengan urutan sebagai berikut:

1. Algoritma mengidentifikasi bahwa karakter terakhir yang berhasil dicocokkan adalah huruf 'A' (indeks pola ke-4).

2. Algoritma mengecek *Failure Table* pada indeks ke-4 tersebut dan menemukan nilai 3, yang mengindikasikan adanya irisan awalan dan akhiran sepanjang tiga karakter.
3. Berdasarkan panduan nilai tersebut, perbandingan karakter selanjutnya dapat langsung dilanjutkan secara presisi antara karakter teks pada indeks ke-5 dengan karakter pola pada indeks ke-3, melompati perbandingan yang tidak perlu.

Lompatan evaluasi semacam ini secara cerdas menghindarkan program dari komputasi berulang. Berkat mekanisme operasional yang stabil, KMP sangat unggul dan cocok diimplementasikan pada sistem yang memproses aliran data masif secara *real-time*. Selain itu, algoritma ini menunjukkan performa yang maksimal ketika dihadapkan pada kasus di mana pola pencariannya memiliki banyak pengulangan *substring*, karena nilai transisi dalam array LPS akan semakin besar dan memfasilitasi rentang lompatan indeks yang jauh lebih efisien.

2.2. Algoritma Boyer-Moore

Algoritma Boyer-Moore merupakan salah satu algoritma *string matching* yang dirancang untuk memberikan performa pencarian yang sangat cepat, melampaui metode konvensional. Keunikan utama sekaligus keunggulan dari algoritma ini dibandingkan algoritma lain adalah arah evaluasinya. Alih-alih mencocokkan karakter dari kiri ke kanan, Boyer-Moore melakukan pencocokan karakter dari kanan ke kiri. Pendekatan ini memungkinkan algoritma untuk melompati banyak karakter pada teks utama secara sekaligus, sehingga mampu mencapai kompleksitas waktu komputasi terbaik secara sub-linear sebesar $O(n/m)$, di mana algoritma semakin cepat seiring dengan bertambah panjangnya pola pencarian. Meskipun demikian, pada kasus terburuk di mana seluruh karakter teks dan pola memiliki format repetitif yang identik, kompleksitas waktunya dapat menyentuh $O(n \times m)$.

Untuk mencapai tingkat lompatan yang efisien, algoritma Boyer-Moore memproses pola pencarian di awal untuk membangun tabel heuristik. Salah satu mekanisme heuristik utama yang digunakan adalah *Bad Character Heuristic*. Tabel *Bad Character* ini bertugas menyimpan informasi mengenai posisi kemunculan terakhir dari setiap karakter yang ada di dalam pola pencarian. Peta informasi inilah yang nantinya digunakan algoritma untuk mengambil keputusan sejauh mana pola harus digeser ketika menemukan karakter pada teks yang tidak cocok (*bad character*).

Secara teknis, alur eksekusi pencarian dengan pendekatan *Bad Character Heuristic* pada Boyer-Moore terbagi menjadi fase prapemrosesan dan fase pencarian dengan langkah-langkah berikut:

- Pada fase prapemrosesan, algoritma menginisialisasi array dengan ukuran sebesar jumlah variasi karakter alfabet. Setiap elemen array diisi dengan indeks kemunculan terakhir dari karakter tersebut pada pola pencarian. Jika karakter tidak ada di dalam pola, nilainya diatur sebagai representasi ketidakhadiran (misalnya -1).
- Pada fase pencarian, algoritma menempatkan pola di awal teks dan mulai membandingkan karakter dari indeks paling kanan dari pola.
- Jika karakter cocok, algoritma memundurkan penunjuk evaluasi ke kiri untuk mengecek karakter sebelumnya hingga seluruh pola dipastikan cocok.

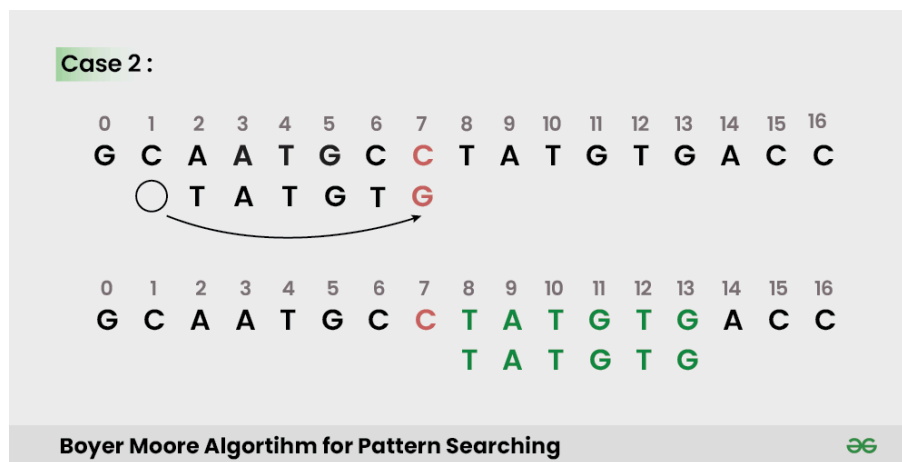
- Jika terjadi ketidakcocokan, algoritma merujuk pada karakter teks yang menyebabkan kegagalan (*bad character*) dan mengecek posisinya di tabel heuristik.
- Apabila *bad character* tersebut terdapat di tempat lain dalam pola pencarian, algoritma menggeser pola ke kanan agar posisi kemunculan terakhir karakter tersebut pada pola sejajar dengan *bad character* pada teks utama.
- Apabila *bad character* tersebut sama sekali tidak terdapat di dalam pola pencarian, algoritma secara agresif menggeser seluruh panjang pola melewati *bad character* tersebut, menghindari perbandingan yang sia-sia.



Gambar 2.2. Ilustrasi Pergeseran Boyer-Moore ketika Karakter Ditemukan pada Pola

<https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>

Penerapan *Bad Character Heuristic* ini dapat dilihat melalui dua skenario utama. Pada skenario pertama (Gambar 2.2), evaluasi dari kanan ke kiri menemukan *mismatch* di mana karakter teks adalah 'A' sementara karakter pola adalah 'G'. Algoritma kemudian mencari kemunculan terakhir huruf 'A' pada pola (yang berada di indeks ke-1). Pola kemudian digeser agar huruf 'A' pada pola sejajar dengan huruf 'A' pada teks utama.



Gambar 2.3. Ilustrasi Pergeseran Boyer-Moore Melewati Karakter yang Tidak Dikenal

<https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>

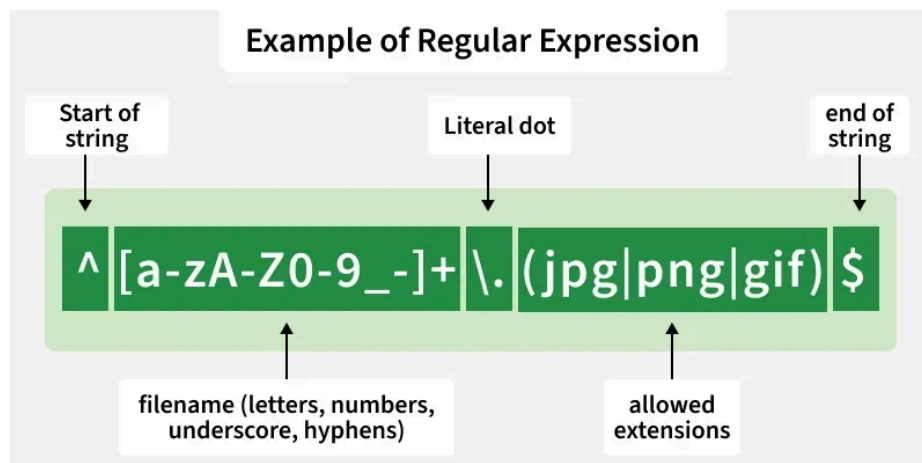
Pada skenario kedua (Gambar 2.3), *mismatch* terjadi pada karakter teks 'C'. Karena huruf 'C' sama sekali tidak eksis di dalam pola pembentuk, algoritma menyimpulkan bahwa setiap pergeseran kecil hanya akan menghasilkan ketidakcocokan berulang. Oleh karena itu, pola langsung digeser melompati karakter 'C' tersebut sepenuhnya.

Mekanisme pergeseran yang sangat masif ini menjadikan algoritma Boyer-Moore sebagai standar industri yang sangat ideal untuk operasi *Search* atau *Find* pada perangkat lunak pengolah kata, seperti *Notepad* atau peramban web. Algoritma ini menunjukkan performa yang sangat maksimal ketika dihadapkan pada teks dengan variasi alfabet yang sangat kaya dan panjang pola pencarian yang cukup besar, karena kondisi tersebut memicu algoritma untuk lebih sering melakukan lompatan jauh melewati teks tanpa perlu melakukan evaluasi karakter demi karakter.

2.3. Regular Expression (Regex)

Regular Expression, atau yang umum disingkat sebagai Regex, merupakan sebuah representasi notasi deklaratif dalam teori bahasa formal yang digunakan untuk mendefinisikan sebuah pola pencarian (*search pattern*) teks. Berbeda dengan algoritma *exact matching* seperti Knuth-Morris-Pratt atau Boyer-Moore yang mencari string statis, Regex dirancang untuk mengevaluasi dan mencocokkan pola string yang dinamis. Pendekatan ini memungkinkan sistem untuk mengidentifikasi serangkaian string yang memiliki variasi format, panjang, atau kombinasi karakter tertentu tanpa harus mendefinisikan setiap kemungkinan secara manual di dalam kode program.

Secara konseptual, sebuah pola Regex dibentuk melalui kombinasi dua jenis karakter utama, yaitu karakter literal dan metakarakter. Karakter literal bertugas untuk dicocokkan secara persis seperti wujud aslinya. Sementara itu, metakarakter bertindak sebagai operator fungsional yang memberikan aturan logika pada pencarian, seperti repetisi, opsionalitas, pengelompokan, atau rentang karakter. Melalui kombinasi ini, sebuah pola tunggal Regex dapat merepresentasikan ribuan kemungkinan variasi kata.



Gambar 2.4. Contoh Pola Regular Expression

<https://www.geeksforgeeks.org/dsa/write-regular-expressions/>

Penerapan kombinasi karakter literal dan metakarakter ini dapat diilustrasikan secara konkret melalui contoh pola RegEx `^[a-zA-Z0-9_-]+\.(jpg|png|gif)$` pada Gambar 2.4, yang dirancang khusus untuk memvalidasi format nama file gambar. Alur pembacaan dan evaluasi notasi tersebut dapat dijabarkan sebagai berikut:

- **Simbol `^`** : Merupakan metakarakter yang menandakan bahwa pencocokan harus dimulai tepat pada awal baris teks.
- **Kurung siku `[...]` dan `+`**: Mendefinisikan *character class*. Rentang `a-zA-Z0-9_-` mengizinkan penggunaan huruf alfabet (besar dan kecil), angka, garis bawah, serta tanda hubung untuk membentuk nama file. Metakarakter `+` setelahnya memastikan bahwa minimal harus ada satu atau lebih karakter dari rentang tersebut yang muncul.
- **Simbol `\.`**: Merupakan teknik pelepasan untuk mengubah fungsi metakarakter titik (yang aslinya berarti "karakter apa saja") menjadi karakter literal titik biasa (*Literal dot*) yang memisahkan nama file dengan ekstensinya.
- **Kurung biasa `(...)` dan `|`**: Berfungsi untuk *grouping*, sementara simbol pita `|` bertindak sebagai operator logika OR. Bagian `(jpg|png|gif)` ini membatasi secara ketat bahwa ekstensi file yang diizinkan hanyalah salah satu dari ketiga format tersebut.
- **Simbol `$`**: Menandakan bahwa pencocokan harus berakhir tepat setelah ekstensi file, mencegah adanya karakter tambahan yang tidak valid di akhir baris.

Kemampuan RegEx dalam mengekspresikan fleksibilitas pola secara padat menjadikannya instrumen yang sangat esensial dalam pemrosesan teks tingkat lanjut. Dalam konteks ekstraksi informasi, RegEx sering kali dialokasikan untuk mendeteksi anomali penulisan atau format khusus yang sengaja disamarkan oleh pengguna, seperti penggantian huruf dengan angka atau penambahan simbol tertentu di tengah kata. Efisiensinya dalam menyelesaikan tantangan *pattern matching* dinamis menjadikannya solusi komputasi yang jauh lebih praktis dan terstruktur dibandingkan dengan menyusun ratusan logika kondisi konvensional.

2.4. Weighted Levenshtein Distance

Levenshtein Distance merupakan sebuah metrik dalam teori informasi dan ilmu komputer yang digunakan untuk mengukur tingkat perbedaan jarak antara dua buah string. Secara konseptual, metrik ini menghitung jumlah minimum operasi modifikasi karakter tunggal yang mutlak diperlukan untuk mengubah satu string menjadi string lainnya. Terdapat tiga jenis operasi fundamental yang diizinkan dalam algoritma ini, yaitu operasi penyisipan (*insertion*), penghapusan (*deletion*), dan penggantian (*substitution*).

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if head}(a) = \text{head}(b), \\ 1 + \min \begin{cases} \text{lev}(\text{tail}(a), b) \\ \text{lev}(a, \text{tail}(b)) \\ \text{lev}(\text{tail}(a), \text{tail}(b)) \end{cases} & \text{otherwise} \end{cases}$$

Gambar 2.5. Definisi Matematis Fungsi Rekursif Levenshtein Distance

https://wikimedia.org/api/rest_v1/media/math/render/svg/d1367dec4582e7b945fcb976bdf94263263e07e

Secara matematis, jarak Levenshtein antara dua string didefinisikan melalui sebuah fungsi rekursif seperti yang diilustrasikan pada Gambar 2.5. Fungsi ini menetapkan kondisi basis di mana jika salah satu string kosong, maka jarak editnya adalah ekuivalen dengan panjang string yang tersisa. Pada tahap komputasi rekursif, apabila karakter yang sedang dievaluasi pada kedua string adalah identik, operasi dilanjutkan tanpa penambahan biaya. Namun, apabila terjadi perbedaan, algoritma akan mengkalkulasi dan mengambil nilai minimum dari tiga kemungkinan operasi dan menambahkan biaya penalti sebesar 1. Sebagai contoh, untuk mengubah kata "kitten" menjadi "sitting", diperlukan jarak minimum sebesar 3 operasi (mengganti 'k' dengan 's', mengganti 'e' dengan 'i', dan menyisipkan 'g' di akhir kalimat).

Dalam penerapannya untuk pemrosesan teks tingkat lanjut, algoritma ini dikembangkan menjadi Weighted Levenshtein Distance. Sebagai bentuk modifikasi dari metrik standar, pendekatan *weighted* memperkenalkan fleksibilitas dengan memungkinkan alokasi biaya penalti yang berbeda-beda untuk setiap jenis operasi, atau bahkan disesuaikan berdasarkan karakter spesifiknya. Sebagai contoh, operasi penggantian huruf 'a' dengan angka '4' dapat dikonfigurasi agar memiliki biaya penalti yang jauh lebih rendah dibandingkan mengganti huruf 'a' dengan 'z', mengingat adanya kemiripan visual di antara keduanya.

Dari segi komputasi, penerapan langsung dari definisi fungsi rekursif naif akan menghasilkan inefisiensi yang ekstrem akibat kalkulasi sub-masalah yang berulang. Oleh karena itu, implementasi algoritma ini di dunia nyata diselesaikan menggunakan pendekatan *Dynamic Programming* seperti algoritma Wagner-Fischer. Algoritma ini memetakan komputasi dengan membangun sebuah matriks berukuran dimensi $m \times n$ secara iteratif dari bawah ke atas (*bottom-up*). Setiap sel pada matriks menyimpan jarak minimum dari *prefix* kedua string, sehingga hasil akhir tingkat kemiripan dapat langsung diekstraksi pada sel ujung matriks dengan kompleksitas waktu yang stabil sebesar $O(m \times n)$.

Karakteristik komputasi inilah yang menjadikan Weighted Levenshtein Distance sangat esensial sebagai mekanisme pencarian berbasis perkiraan (*fuzzy string searching*). Dalam arsitektur *pattern matching*, algoritma ini diposisikan sebagai *fallback*. Ketika algoritma *exact match* gagal menemukan target akibat adanya modifikasi teks, kamuflase kata, atau *typo*, Weighted Levenshtein mengambil alih untuk memvalidasi tingkat kemiripan kata, memastikan sistem tetap adaptif dalam mendeteksi pola yang telah dimanipulasi oleh pengguna.

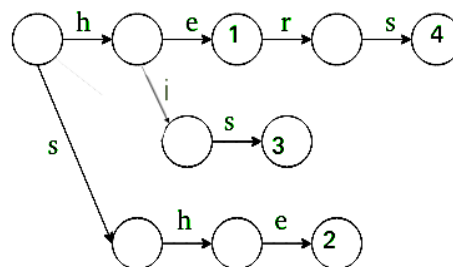
2.5. Algoritma Aho-Corasick

Algoritma Aho-Corasick merupakan algoritma *pattern matching* tingkat lanjut yang dirancang khusus untuk menyelesaikan permasalahan pencarian multikata (*multi-pattern searching*). Berbeda dengan KMP atau Boyer-Moore yang secara mendasar didesain untuk mencari satu kata kunci spesifik dalam satu waktu, Aho-Corasick memiliki kapabilitas untuk mencari ratusan bahkan ribuan kata kunci secara serentak di dalam sebuah teks hanya dengan melakukan satu kali pemindaian jalan. Efisiensi ini menghasilkan kompleksitas waktu pencarian yang sangat optimal, yaitu $O(n + m + z)$, di mana n adalah panjang teks utama, m adalah total jumlah karakter dari seluruh kata kunci gabungan, dan z adalah jumlah total kecocokan yang ditemukan.

Kunci utama dari efisiensi Aho-Corasick terletak pada fase prapemrosesannya yang cerdas, di mana seluruh kata kunci dikompilasi ke dalam sebuah struktur data *Finite Automaton*. Automata ini dibangun di atas fondasi struktur data *Trie* dan dikendalikan oleh tiga fungsi transisi utama, yaitu fungsi *Goto*, *Failure*, dan *Output*.

- **Fungsi Goto:** Merupakan representasi langsung dari struktur *Trie* dasar. Fungsi ini memetakan transisi normal dari satu *state* ke *state* berikutnya berdasarkan karakter yang sedang dibaca.
- **Fungsi Failure:** Berperan layaknya array LPS pada algoritma KMP, namun diimplementasikan pada skala *Tree*. Fungsi ini menyimpan tautan yang mengarahkan sebuah *state* ke *state* lain jika terjadi ketidakcocokan karakter. Tautan ini mengarah pada *proper suffix* terpanjang dari *state* saat ini yang juga merupakan *proper prefix* dari kata kunci lain di dalam *Trie*, memastikan algoritma tidak pernah mundur saat menyisir teks.
- **Fungsi Output:** Bertugas sebagai penanda. Fungsi ini menyimpan indeks atau daftar kata kunci yang telah berhasil dicocokkan sepenuhnya ketika mesin mencapai suatu *state* tertentu.

Trie for Arr[] = { he , she, his , hers }

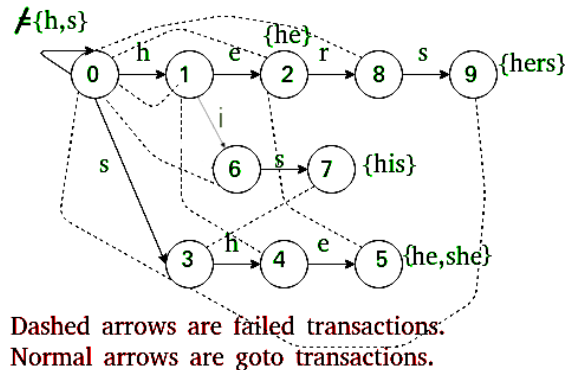


Gambar 2.6. Pembentukan Fungsi *Goto* dalam Struktur *Trie*

<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>

Secara teknis, pembentukan mesin pencari Aho-Corasick diawali dengan membangun struktur *Trie* dari sekumpulan kata kunci. Pada tahap ini, fungsi *Goto* terbentuk secara hierarkis, sementara fungsi *Output* ditandai pada simpul-simpul terminal dari setiap kata kunci. Setelah *Trie* dasar terbentuk,

algoritma melakukan *Breadth-First Traversal* untuk mengalkulasi dan memasang tautan-tautan fungsi *Failure* pada setiap simpul.



Gambar 2.7. Penggabungan Fungsi *Goto* dan *Failure* Membentuk Automata

<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>

Pada saat yang bersamaan, algoritma juga melakukan penggabungan nilai-nilai fungsi *Output* berdasarkan alur tautan *Failure* tersebut (Gambar 2.7). Penggabungan ini krusial untuk menangani kasus di mana sebuah kata kunci (misal: "he") merupakan bagian *substring* utuh dari kata kunci lain yang lebih panjang (misal: "she").

Setelah mesin automata selesai dikonstruksi, fase pencarian dieksekusi dengan sangat lugas. Algoritma membaca karakter dari teks utama satu per satu secara sekuensial. Untuk setiap karakter, mesin mencoba berpindah ke *state* selanjutnya menggunakan fungsi *Goto*. Jika rute *Goto* terputus (mendeteksi *mismatch*), mesin akan merujuk pada fungsi *Failure* untuk bergeser ke cabang *Trie* lain secara presisi tanpa memundurkan kursor baca pada teks utama. Setiap kali mesin menginjak sebuah *state* yang memiliki nilai pada fungsi *Output*, program akan secara instan mencatat posisi penemuan kata kunci tersebut.

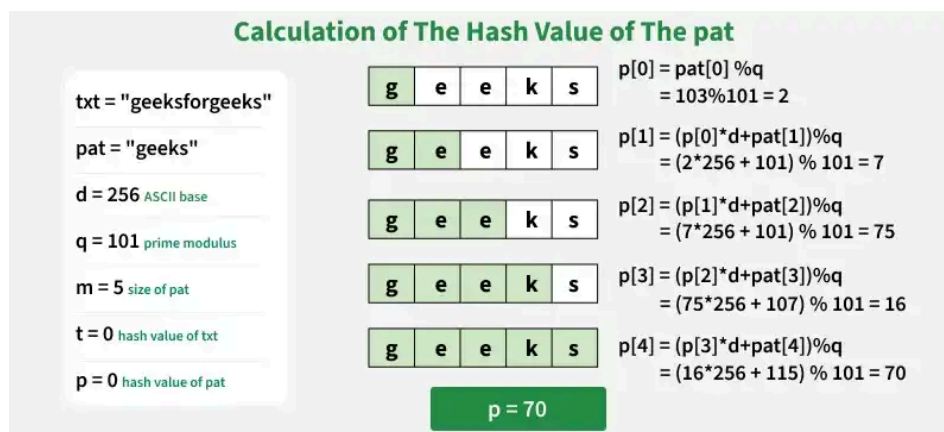
Kapasitas Aho-Corasick dalam memproses ribuan pola pencarian tanpa degradasi performa pada fase pemindaian teks menjadikannya instrumen yang tidak tergantikan untuk aplikasi industri berskala masif. Algoritma ini merupakan standar emas di balik berbagai sistem kritis, mulai dari perangkat lunak pendeteksi plagiarisme otomatis, mesin ekstraksi data teks, sistem deteksi intrusi jaringan yang memindai ribuan *signature* virus secara *real-time*, hingga pencarian untaian basa nitrogen yang kompleks dalam bidang bioinformatika.

2.6. Algoritma Rabin-Karp

Algoritma Rabin-Karp menawarkan pendekatan yang secara fundamental berbeda dari algoritma *pattern matching* konvensional seperti Knuth-Morris-Pratt atau Boyer-Moore. Alih-alih melakukan komputasi perbandingan karakter demi karakter secara langsung pada setiap pergeseran, algoritma ini mentransformasikan permasalahan pencocokan string menjadi permasalahan pencocokan nilai numerik.

Rabin-Karp memanfaatkan fungsi *hash* untuk membandingkan kecocokan antara pola pencarian dengan *substring* dari teks utama yang sedang dievaluasi. Pada kasus rata-rata, pendekatan komputasi numerik ini mampu memberikan kompleksitas waktu pencarian sebesar $O(n + m)$.

Kunci utama dari efisiensi algoritma Rabin-Karp adalah penggunaan teknik *Rolling Hash*. Jika algoritma *naive* menghitung nilai *hash* dari awal untuk setiap *substring* teks yang dipindai, teknik *Rolling Hash* memungkinkan sistem untuk memperbarui nilai *hash* secara dinamis dan konstan pada setiap pergeseran *sliding window*. Hal ini dicapai secara matematis dengan menghapus kontribusi nilai karakter pertama yang keluar dari jendela pembacaan, menggeser sisa nilai, dan menambahkan kontribusi nilai karakter baru yang masuk ke dalam jendela pembacaan.



Gambar 2.8. Ilustrasi Kalkulasi Nilai *Hash* pada Algoritma Rabin-Karp

<https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>

Secara teknis, proses komputasi diawali dengan menginisialisasi basis perhitungan (umumnya menggunakan nilai 256 yang merepresentasikan total variasi karakter ASCII) dan menentukan sebuah bilangan prima berukuran besar sebagai pembagi (*modulus*). Penggunaan operasi *modulo* bilangan prima ini sangat krusial untuk menjaga agar nilai komputasi tidak melampaui batas kapasitas memori integer (*integer overflow*), sekaligus menekan probabilitas terjadinya bentrokan nilai *hash* (*hash collision*). Pada fase awal, algoritma akan mengalkulasi nilai *hash* dari pola pencarian dan membandingkannya dengan nilai *hash* dari *substring* pertama pada teks utama yang panjangnya sama dengan panjang pola.

Apabila kedua nilai *hash* tersebut tidak sama, dapat dipastikan secara matematis bahwa kedua string tersebut berbeda, sehingga algoritma langsung menggeser jendela pembacaan dan melakukan fungsi *Rolling Hash* untuk mendapatkan nilai numerik dari *substring* selanjutnya.

Namun, apabila nilai *hash* antara *substring* teks dan pola pencarian menunjukkan hasil yang sama, kecocokan pola belum dapat dikonfirmasi secara mutlak. Hal ini disebabkan oleh sifat dasar dari operasi *modulo* yang berpotensi menghasilkan *hash collision*, di mana dua string dengan komposisi karakter yang sama sekali berbeda dapat menghasilkan nilai *hash* yang ekuivalen. Oleh karena itu, setiap kali nilai *hash* cocok, algoritma diwajibkan untuk melakukan tahap verifikasi akhir dengan membandingkan *substring*

dan pola secara karakter demi karakter. Jika verifikasi karakter ini juga cocok, maka kemunculan pola dikonfirmasi.

Kewajiban untuk melakukan verifikasi sekunder inilah yang menjadi titik kelemahan komputasional dari algoritma Rabin-Karp. Pada skenario kasus terburuk di mana terjadi *hash collision* secara terus-menerus di hampir setiap pergeseran (misalnya karena pemilihan bilangan prima *modulus* yang buruk atau struktur teks repetitif yang ekstrem), algoritma terpaksa melakukan evaluasi manual berulang kali. Kondisi ini mendegradasi performa algoritma secara drastis, menjatuhkan kompleksitas waktunya hingga menyentuh batas $O(n \times m)$, yang mana setara dengan kecepatan metode *naive brute-force*. Terlepas dari kelemahan pada *worst-case* tersebut, arsitektur berbasis nilai *hash* dari Rabin-Karp menjadikannya teknik yang sangat diandalkan dalam permasalahan deteksi duplikasi *substring* jamak, seperti pada perangkat lunak pendeteksi plagiarisme dokumen.

2.7. Chromium Browser Extension

Chromium Browser Extension merupakan perangkat lunak berukuran kecil yang dirancang untuk memodifikasi, mengotomatisasi, dan meningkatkan fungsionalitas peramban web berbasis Chromium. Berbeda dengan aplikasi *desktop* mandiri, ekstensi peramban dikembangkan sepenuhnya menggunakan tumpukan teknologi web standar, yakni HTML, CSS, dan JavaScript. Ekstensi ini berjalan di dalam lingkungan eksekusi terisolasi yang disediakan oleh peramban, sehingga memungkinkan pengguna untuk menyesuaikan pengalaman menjelajah web mereka dengan menambahkan fitur-fitur spesifik langsung ke dalam antarmuka peramban.

Arsitektur dasar dari sebuah ekstensi Chromium dikendalikan secara terpusat oleh sebuah berkas konfigurasi utama yang disebut `manifest.json`. Berkas manifest ini bertindak sebagai cetak biru yang mendefinisikan metadata ekstensi, mengatur hak akses atau *permissions*, serta mendaftarkan skrip-skrip komponen yang menyusun ekstensi tersebut. Dalam konteks aplikasi pemrosesan teks yang berinteraksi langsung dengan konten situs web, komponen paling krusial yang digunakan adalah *Content Script*. *Content Script* merupakan berkas JavaScript khusus yang diinjeksikan dan dieksekusi secara langsung di dalam ruang lingkup halaman web yang sedang dikunjungi oleh pengguna.

Melalui *Content Script* inilah, ekstensi peramban mendapatkan hak akses untuk berinteraksi dengan *Document Object Model* (DOM) halaman web target. Kemampuan interaksi DOM ini menjadi fondasi utama dalam operasional algoritma *pattern matching*. Sistem ekstensi akan memindai halaman untuk mengekstrak elemen-elemen simpul teks, mengirimkan data tersebut ke fungsi algoritma pencarian di dalam memori, dan kemudian melakukan manipulasi struktur HTML secara dinamis berdasarkan hasil temuan.

Manipulasi ini memungkinkan ekstensi untuk menyisipkan elemen HTML baru beserta pengaturan gaya dan atribut kustom tepat di atas teks target. Dengan memanfaatkan arsitektur ekstensi Chromium ini, pemrosesan dan modifikasi antarmuka dapat dilakukan secara *real-time* tanpa merusak fungsi bawaan maupun struktur tampilan asli dari situs web yang sedang dipindai.

Bab III

Analisis Pemecahan Masalah

3.1. Analisis Masalah dan Kebutuhan Sistem

Maraknya penyebaran situs perjudian daring di ruang digital sering kali memanfaatkan celah pada platform web yang sah untuk menyisipkan konten promosi secara terselubung. Pihak tidak bertanggung jawab secara aktif melakukan penyamaran kata kunci (*keyword camouflage*) untuk menghindari deteksi sistem. Penyamaran ini dilakukan melalui manipulasi karakter, penyisipan simbol tidak relevan, hingga menanamkan teks promosi tersebut di dalam aset visual (gambar atau *banner*). Mengingat teknik penyamaran yang semakin dinamis, pemindaian teks konvensional maupun pengawasan manual menjadi sangat tidak efisien dan rentan terhadap kelalaian.

Berdasarkan analisis permasalahan tersebut, diperlukan sebuah perangkat lunak otomatis yang mampu mengidentifikasi dan memitigasi paparan konten bermasalah secara menyeluruh di latar belakang. Untuk menjawab kebutuhan tersebut, dibangunlah sebuah ekstensi peramban berbasis Chromium yang dilengkapi dengan serangkaian kebutuhan fungsional utama sebagai berikut:

- **Implementasi Kolaborasi Multi-Algoritma:** Sistem tidak hanya bergantung pada satu metode pencarian, melainkan mengintegrasikan berbagai algoritma *pattern matching* secara sinergis. Algoritma *exact matching* (seperti KMP, Boyer-Moore, Aho-Corasick, dan Rabin-Karp) bertugas mengidentifikasi kata kunci mutlak, sementara pendekatan *regex matching* dan *fuzzy matching* bertindak sebagai lapis pertahanan untuk menangkap kata kunci yang telah dimanipulasi dengan modifikasi karakter tingkat tinggi.
- **Mekanisme Sensor dan Highlighter:** Sebagai langkah perlindungan langsung terhadap pengguna, setiap teks bermasalah yang berhasil terdeteksi tidak hanya akan diberikan penyorotan visual (*highlight*) sebagai bentuk peringatan, tetapi juga dapat dikenakan mekanisme sensor (menyembunyikan atau mengganti teks target) agar konten negatif tidak terpapar langsung.
- **Optical Character Recognition (OCR):** Menyadari bahwa ancaman promosi sering kali beralih ke medium gambar, sistem dilengkapi dengan kapabilitas OCR. Fitur ini memungkinkan ekstensi untuk memindai aset gambar yang dimuat pada halaman web, mengekstrak teks yang tertanam di dalamnya, dan memprosesnya melalui mesin pencari algoritma.
- **Informasi Metrik Interaktif (Tooltip):** Saat pengguna mengarahkan kursor (*hover*) ke atas elemen yang terindikasi atau telah disensor, ekstensi akan memunculkan jendela informasi mengambang (*tooltip*). Jendela ini memberikan transparansi sistem dengan menampilkan algoritma spesifik yang berhasil melakukan deteksi, metrik waktu eksekusi komputasi, serta jumlah kemunculan *keyword*.
- **Antarmuka Popup Analitik:** Ekstensi menyediakan dasbor antarmuka *popup* interaktif yang memuat informasi statistik secara terpusat. Melalui antarmuka ini, pengguna dapat melihat rekapitulasi data perbandingan performa algoritma secara komprehensif, mencakup jumlah perbandingan karakter dan total waktu komputasi yang dihabiskan oleh masing-masing algoritma, memenuhi kebutuhan analisis spesifikasi sistem.

3.2. Strategi Pemrosesan Teks dan Pemetaan Algoritma

Permasalahan utama pada sistem Judol Detector adalah mendeteksi keberadaan konten yang mengandung indikasi judi online pada halaman web secara otomatis. Konten tersebut dapat muncul dalam berbagai bentuk, mulai dari teks biasa seperti “slot”, “gacor”, dan “maxwin”, variasi gabungan kata dan angka, misalnya “GACOR99”, hingga teks yang disamarkan dengan simbol lain seperti “sl0t gac0r”. Oleh karena itu, proses pemecahan masalah diawali dengan memetakan halaman web menjadi kumpulan data teks yang dapat diproses oleh algoritma pencocokan string.

Pada tahap awal, ekstensi mengambil seluruh text node dari *Document Object Model* (DOM) pada halaman web. *Text node* dipilih karena konten yang terlihat oleh pengguna pada halaman web umumnya direpresentasikan sebagai node teks di dalam elemen HTML. Namun, tidak semua teks pada DOM perlu dianalisis. Teks yang berada di dalam elemen seperti *script*, *style*, *input*, dll diabaikan karena tidak merepresentasikan konten utama yang dibaca pengguna atau dapat menimbulkan hasil deteksi yang tidak relevan. Setelah *text node* dikumpulkan, setiap teks dinormalisasi, terutama dengan mengubah huruf menjadi lowercase. Normalisasi ini bertujuan agar pencocokan tidak bergantung pada kapitalisasi, sehingga kata seperti “SLOT”, “Slot”, dan “slot” diperlakukan sebagai bentuk yang sama.

Setelah tahap praproses selesai, sistem menjalankan beberapa strategi pencocokan string. Untuk pencocokan exact matching, sistem menggunakan algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore (BM). Kedua algoritma tersebut digunakan untuk mencari kemunculan kata kunci yang tersimpan pada file keyword.txt di dalam teks halaman web. Kata kunci pada file tersebut menjadi pattern, sedangkan isi text node pada halaman web menjadi text. Dengan pemetaan ini, persoalan deteksi konten judi online dapat direduksi menjadi persoalan pencarian pattern di dalam text.

Pada algoritma KMP, setiap keyword diproses sebagai pattern untuk membentuk tabel Longest Prefix Suffix (LPS). Tabel LPS menyimpan informasi mengenai panjang prefix terpanjang yang juga merupakan suffix pada setiap posisi pattern. Informasi ini digunakan ketika terjadi mismatch antara pattern dan text, sehingga algoritma tidak perlu mengulang pencocokan dari awal. Dengan demikian, KMP cocok digunakan untuk melakukan pencarian exact matching secara sistematis karena setiap karakter pada text dapat diproses secara efisien tanpa backtracking yang berlebihan. Dalam konteks Judol Detector, setiap text node dibandingkan dengan seluruh keyword menggunakan KMP, lalu setiap posisi kemunculan keyword dicatat sebagai hasil deteksi.

Sementara itu, algoritma Boyer-Moore memetakan masalah pencocokan string dengan pendekatan berbeda. BM mencocokkan pattern dari kanan ke kiri menggunakan teknik looking-glass. Ketika terjadi mismatch, algoritma menggunakan character-jump heuristic berdasarkan last occurrence table. Tabel ini menyimpan posisi kemunculan terakhir setiap karakter pada pattern. Jika karakter pada text yang menyebabkan mismatch masih terdapat pada pattern, pattern digeser agar kemunculan terakhir karakter tersebut sejajar dengan posisi mismatch. Jika tidak memungkinkan, pattern digeser minimal satu posisi. Jika karakter tersebut tidak terdapat pada pattern, pattern dapat digeser lebih jauh. Strategi ini

memungkinkan BM melewati beberapa posisi text yang tidak mungkin menghasilkan match, sehingga pada banyak kasus dapat bekerja lebih cepat dibanding pencocokan langsung dari kiri ke kanan.

Dalam implementasi sistem, KMP dan BM sama-sama dijalankan terhadap text node yang telah dinormalisasi. Setiap hasil pencocokan menyimpan informasi berupa keyword yang terdeteksi, algoritma yang menemukan match, indeks awal dan akhir match pada text node, serta jumlah perbandingan karakter. Informasi indeks tersebut kemudian digunakan untuk menandai bagian teks yang terdeteksi pada DOM. Dengan cara ini, hasil algoritma tidak hanya berupa status ditemukan atau tidak ditemukan, tetapi juga dapat dipetakan kembali ke posisi asli pada halaman web untuk keperluan highlight dan tooltip.

Selain exact matching, sistem juga menjalankan pencocokan berbasis regular expression untuk mendeteksi pola kata yang diikuti angka, seperti “GACOR99”, “MAXWIN88”, dan “SLOT88”. Pencocokan ini tidak bergantung pada daftar keyword karena tujuannya adalah menangkap pola umum yang sering digunakan pada konten judi online. Setelah itu, sistem dapat menjalankan fuzzy matching menggunakan Weighted Levenshtein Distance pada bagian teks yang belum tertangani oleh exact matching. Fuzzy matching digunakan untuk mendeteksi variasi penulisan yang menyerupai keyword, misalnya penggantian huruf dengan angka yang mirip secara visual seperti O menjadi 0, A menjadi 4, I menjadi 1, E menjadi 3, dan S menjadi 5.

Secara keseluruhan, strategi pemrosesan teks pada Judol Detector dilakukan dengan mengubah halaman web menjadi kumpulan text node, menormalisasi teks, menjalankan algoritma pencocokan string, lalu memetakan kembali hasil pencocokan ke DOM. Pemetaan masalah ke algoritma KMP dan BM dilakukan dengan menjadikan keyword sebagai pattern dan teks halaman sebagai text. KMP digunakan untuk pencarian exact matching dengan bantuan tabel LPS, sedangkan BM digunakan untuk pencarian exact matching dari kanan ke kiri dengan bantuan last occurrence table. Kombinasi kedua algoritma ini memungkinkan sistem mendeteksi keyword judi online secara akurat, sekaligus menyediakan data perbandingan performa antaralgoritma melalui jumlah match, waktu eksekusi, dan jumlah perbandingan karakter.

3.3. Arsitektur Extension

Judol Detector dibangun dengan TypeScript dan Vite sebagai build tool, menghasilkan dua entry point terpisah: content script (content.js) yang berjalan pada setiap halaman web, dan popup script (popup.js) untuk antarmuka statistik. Manifest versi 3 mendefinisikan permission storage, host_permissions ke semua URL, dan mendaftarkan content script yang berjalan pada document_idle. Komponen-komponen utama extension beserta perannya adalah sebagai berikut:

- **content.ts:** Orchestrator utama content script. Membaca keyword.txt, mengumpulkan text node, menjalankan semua algoritma secara berurutan, mengumpulkan hasil, lalu memanggil highlighter dan menyimpan statistik ke chrome.storage.local.
- **dom.ts:** Mengumpulkan text node dari DOM menggunakan createTreeWalker(). Melewati tag SCRIPT, STYLE, NOSCRIPT, TEXTAREA, INPUT, SELECT, dan OPTION agar tidak mendeteksi kode atau nilai input.

- **highlighter.ts**: Menerima daftar DomMatchResult, menghitung batas-batas (boundary) irisan semua match pada setiap text node, lalu memecah teks dan membungkus bagian yang cocok dengan elemen `<mark class="judol-highlight">`. Mendukung overlap dan multiple algorithm pada satu elemen.
- **tooltip.ts**: Membuat elemen tooltip custom DOM (`<div id="ext-judol-tooltip">`) dan melakukan binding event mouseenter/mousemove/mouseleave pada setiap elemen mark. Tooltip menampilkan jenis matching (Exact/Regex/Fuzzy), algoritma, keyword, jumlah kemunculan, dan waktu eksekusi.
- **cursor.ts**: Menangani fitur blur teks (bonus). Mencari ancestor block element terdekat dari setiap match node melewati tag inline (EM, STRONG, A, B, I, CODE) dan menghentikan pencarian di tag immune (BODY, HTML, MAIN, ARTICLE, SECTION), lalu menambahkan class `judol-container-blur` untuk mengaplikasikan blur CSS.
- **popup.ts**: Membaca statistik dari `chrome.storage.local` dan merender grafik batang (bar chart) perbandingan keyword serta tabel performa algoritma (nama, jumlah match, waktu eksekusi, jumlah perbandingan). Menyediakan toggle untuk Blur Teks, Aho-Corasick, dan Rabin-Karp yang tersinkronisasi secara realtime melalui `chrome.storage.onChanged`.

3.4. Contoh Ilustrasi Kasus

Untuk menggambarkan alur kerja sistem Judol Detector, misalkan terdapat sebuah halaman web dengan potongan konten sebagai berikut:



*ceritanya gambar

SLOT GACOR99
GAMPANG MAXWIN

Temukan situs slot gacor terbaik hari ini.
Promo besar MAXWIN88 tersedia untuk member baru.
Main di slot sekarang dan dapatkan bonus menarik.

Kasus tersebut menunjukkan beberapa bentuk konten yang perlu ditangani oleh sistem. Kalimat pertama mengandung keyword yang ditulis secara normal, yaitu slot dan gacor. Kalimat kedua mengandung pola kata yang diikuti angka, yaitu MAXWIN88. Kalimat ketiga mengandung variasi penulisan slot, yaitu bentuk manipulasi dari kata slot dengan mengganti huruf o menjadi angka 0. Sementara itu, gambar banner mengandung teks yang tidak dapat dibaca langsung dari DOM, sehingga perlu diproses menggunakan OCR.

Pada tahap pertama, ekstensi mengambil seluruh text node dari halaman web. Setelah text node dikumpulkan, teks dinormalisasi menjadi lowercase agar pencocokan tidak bergantung pada kapitalisasi. Dengan demikian, teks seperti MAXWIN88 tetap dapat diproses meskipun menggunakan huruf kapital. Misalnya, text node pertama: “Temukan situs slot gacor terbaik hari ini.”. Dinormalisasi menjadi: “temukan situs slot gacor terbaik hari ini.”.

Selanjutnya, sistem menjalankan exact matching menggunakan algoritma KMP dan Boyer-Moore terhadap keyword yang terdapat pada keyword.txt. Jika slot dan gacor terdapat di dalam daftar keyword, maka kedua algoritma akan menemukan kemunculan kata tersebut pada text node. Dalam pemetaan algoritma, teks halaman menjadi text, sedangkan setiap keyword menjadi pattern. Untuk keyword slot, algoritma mencari kemunculan pattern slot di dalam text. Ketika ditemukan kecocokan, sistem menyimpan indeks awal dan akhir match agar bagian teks tersebut dapat diberi highlight pada DOM.

Hasil match dari KMP dan Boyer-Moore kemudian dikumpulkan sebagai hasil exact matching. Jika kedua algoritma menemukan keyword yang sama pada lokasi yang sama, sistem tetap dapat mencatat bahwa match tersebut ditemukan oleh kedua algoritma. Informasi ini digunakan pada tooltip untuk menampilkan algoritma yang terlibat serta statistik pencarian.

Pada tahap berikutnya, sistem menjalankan regex matching untuk menangkap pola <kata><angka>. Pada kalimat: “Promo besar MAXWIN88 tersedia untuk member baru.”. Regex akan mendeteksi MAXWIN88 karena pola tersebut terdiri atas kata yang langsung diikuti oleh angka. Pencocokan regex tidak bergantung pada daftar keyword di keyword.txt, melainkan pada pola umum yang sering digunakan pada konten judi online. Dengan demikian, meskipun MAXWIN88 tidak tercantum sebagai keyword exact, sistem tetap dapat menandainya karena memenuhi pola regex.

Tahap berikutnya adalah fuzzy matching menggunakan Weighted Levenshtein Distance. Fuzzy matching digunakan untuk mendeteksi teks yang mirip dengan keyword, tetapi tidak sama persis. Misalnya pada kalimat: “Main di slot sekarang dan dapatkan bonus menarik.”. Kata slot tidak sama persis dengan keyword slot karena huruf o diganti dengan angka 0. Exact matching tidak akan menemukan kecocokan karena karakter yang dibandingkan berbeda. Namun, Weighted Levenshtein Distance memberikan biaya substitusi yang lebih kecil untuk karakter yang mirip secara visual, seperti o dan 0. Oleh karena itu, jarak antara slot dan slot menjadi kecil, sehingga nilai kemiripannya tetap tinggi dan dapat melewati threshold fuzzy matching.

Selain teks DOM, sistem juga dapat memproses gambar menggunakan OCR. Pada gambar banner dengan tulisan: “SLOT GACOR99”. Karena teks tersebut berada di dalam gambar, text node pada DOM tidak memuat kata SLOT maupun GACOR99. Oleh karena itu, sistem terlebih dahulu mengambil elemen

gambar yang memenuhi kriteria pemrosesan, seperti sudah selesai dimuat dan memiliki ukuran minimum tertentu. Gambar tersebut kemudian diproses menggunakan Tesseract.js untuk mengekstraksi teks visual menjadi string biasa.

Jika OCR Berhasil membaca gambar sebagai “*SLOT GACOR99*”, maka teks hasil OCR tersebut diproses kembali menggunakan algoritma deteksi teks. Exact matching dapat mendeteksi keyword slot dan gacor, sedangkan regex dapat mendeteksi pola jika terdapat gabungan kata dan angka, misalnya *GACOR99*. Jika ditemukan konten yang sesuai, gambar diberi censorship berupa blur. Dengan demikian, sistem tidak hanya mampu menangani teks biasa pada DOM, tetapi juga konten promosi judi online yang sengaja disisipkan dalam bentuk gambar.

Setelah seluruh proses deteksi selesai, setiap hasil match dipetakan kembali ke elemen DOM yang bersesuaian. Untuk teks, sistem membungkus bagian yang terdeteksi dengan elemen highlight sehingga pengguna dapat melihat bagian yang dianggap mencurigakan. Untuk gambar, sistem menerapkan efek blur pada elemen gambar yang terdeteksi. Saat pengguna mengarahkan kursor ke hasil deteksi, sistem menampilkan tooltip yang memuat informasi keyword, algoritma yang digunakan, jumlah kemunculan, dan waktu eksekusi.

Dengan ilustrasi ini, terlihat bahwa setiap algoritma memiliki peran berbeda dalam menyelesaikan masalah. KMP dan Boyer-Moore digunakan untuk mendeteksi keyword yang muncul secara tepat. Regex digunakan untuk mendeteksi pola promosi yang menggabungkan kata dan angka. Weighted Levenshtein Distance digunakan untuk mendeteksi variasi penulisan yang dimanipulasi. OCR digunakan untuk memperluas cakupan deteksi ke konten berbasis gambar. Kombinasi beberapa strategi tersebut membuat Judol Detector mampu menangani berbagai bentuk kemunculan konten judi online pada halaman web.

Bab IV

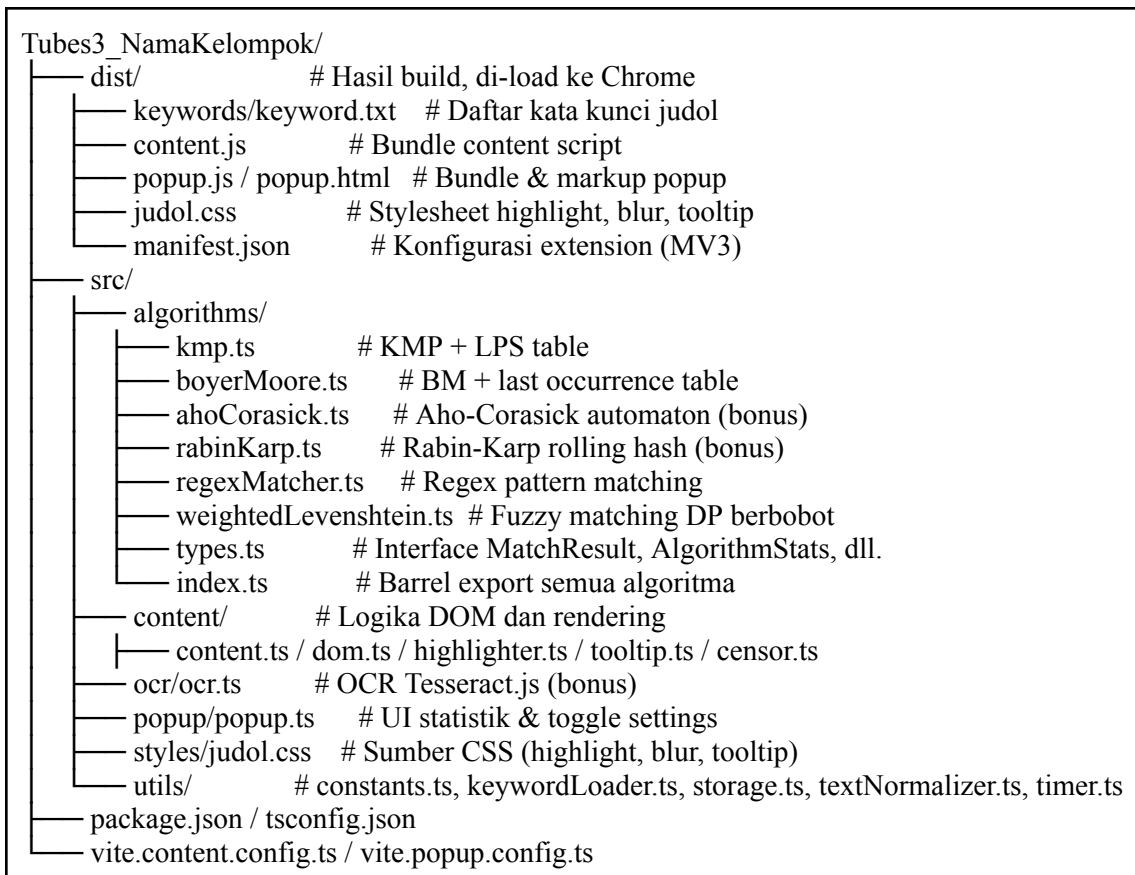
Implementasi dan Pengujian

4.1. Implementasi

Program ini diimplementasikan sebagai Chromium browser extension berbasis TypeScript yang di-build menggunakan Vite. Extension terdiri dari dua komponen utama: content script yang diinjeksikan ke setiap halaman web untuk melakukan deteksi dan highlighting, serta popup script yang menyediakan antarmuka statistik dan pengaturan bagi pengguna.

4.1.1. Struktur Program

Berikut adalah struktur direktori lengkap dari repositori Tubes3_NamaKelompok:



4.1.2. Implementasi Struktur Data

Pada implementasi program, struktur data digunakan untuk menyimpan hasil pencarian, statistik algoritma, data text node dari DOM, konfigurasi extension, serta hasil OCR pada gambar. Penggunaan struktur data dibuat berbasis interface TypeScript agar setiap modul memiliki format data yang konsisten dan mudah diproses oleh modul lain, seperti highlighter, tooltip, popup, dan storage.

Struktur data utama untuk hasil deteksi didefinisikan melalui `MatchResult`. Struktur ini menyimpan keyword yang ditemukan, algoritma yang menemukan match, posisi awal dan akhir match pada teks, teks yang cocok, jumlah perbandingan, serta waktu pencarian. Dengan format ini, semua algoritma dapat menghasilkan output dengan bentuk yang sama.

```
export type MatchingAlgorithm =  
  | "KMP"  
  | "Boyer-Moore"  
  | "Regex"  
  | "Weighted Levenshtein"  
  | "Aho-Corasick"  
  | "Rabin-Karp"  
  | "OCR";  
  
export interface MatchResult {  
  keyword: string;  
  algorithm: MatchingAlgorithm;  
  startIndex: number;  
  endIndex: number;  
  matchedText: string;  
  comparisons: number;  
  searchTime: number;  
}  
  
export interface AlgorithmStats {  
  algorithm: MatchingAlgorithm;  
  executionTimeMs: number;  
  matchCount: number;  
  comparisons: number;  
}  
  
export interface DomMatchResult {  
  node: Text;  
  sourceText: string;  
  match: MatchResult;  
}
```

Gambar 4.1.2.1. Struktur `MatchResult`, `AlgorithmStats`, dan `DomMatchResult`

`MatchResult` digunakan pada level algoritma, sedangkan `DomMatchResult` digunakan setelah hasil pencarian dikaitkan kembali dengan node teks asli pada halaman web. Dengan menyimpan referensi `node: Text`, program dapat melakukan manipulasi DOM secara tepat, misalnya memberi highlight pada bagian teks yang terdeteksi.

Untuk merepresentasikan teks halaman, program menggunakan struktur `TextNodeInfo`. Struktur ini menyimpan node teks asli beserta isi teksnya. Data ini diperoleh dari traversal DOM menggunakan `TreeWalker`, kemudian dipakai sebagai input untuk algoritma pencarian.

```
export interface TextNodeInfo {  
  node: Text;  
  text: string;  
}
```

Gambar 4.1.2.2. Struktur `TextNodeInfo`

Setiap algoritma juga memiliki struktur hasil khusus. Pada KMP, hasil per keyword menyimpan tabel LPS, jumlah perbandingan preprocessing, jumlah perbandingan pencarian, total perbandingan, dan daftar match. Struktur ini diperlukan karena KMP memiliki tahap preprocessing sebelum pencarian dilakukan.

```
export interface KmpKeywordResult {  
  keyword: string;  
  lpsTable: number[];  
  preprocessingComparisons: number;  
  searchComparisons: number;  
  totalComparisons: number;  
  matches: MatchResult[];  
}  
  
export interface KmpSearchResult {  
  matches: MatchResult[];  
  keywordResults: KmpKeywordResult[];  
  comparisons: number;  
}
```

Gambar 4.1.2.3. Struktur `KmpKeywordResult` dan `KmpSearchResult`

Pada Boyer-Moore, struktur data penting yang digunakan adalah `Map<string, number>` untuk tabel last occurrence. Struktur `Map` dipilih karena dapat memetakan setiap karakter ke posisi kemunculan terakhirnya dalam pattern.

```
export interface BoyerMooreKeywordResult {  
  keyword: string;  
  lastOccurrence: Map<string, number>;  
  searchComparisons: number;  
  matches: MatchResult[];  
}  
  
export interface BoyerMooreSearchResult {  
  matches: MatchResult[];  
  keywordResults: BoyerMooreKeywordResult[];  
  comparisons: number;  
}
```

Gambar 4.1.2.4. Struktur BoyerMooreKeywordResult dan BoyerMooreSearchResult

Pada Weighted Levenshtein, program menggunakan beberapa struktur data tambahan. WeightedDistanceResult digunakan untuk menyimpan jarak hasil perhitungan dan jumlah perbandingan. TextToken digunakan untuk menyimpan token teks beserta posisi awal dan akhirnya pada string asli. Struktur ini penting karena fuzzy matching dilakukan pada token, bukan langsung pada seluruh teks tanpa segmentasi.

```
interface WeightedDistanceResult {  
  distance: number;  
  comparisons: number;  
}  
  
interface TextToken {  
  text: string;  
  startIndex: number;  
  endIndex: number;  
}  
  
export interface WeightedLevenshteinSearchResult {  
  matches: MatchResult[];  
  comparisons: number;  
}
```

Gambar 4.1.2.5. Struktur WeightedDistanceResult, TextToken dan WeightedLevenshteinSearchResult

Pada algoritma Aho-Corasick, struktur data utama yang digunakan adalah automaton berbentuk trie dengan failure link. Setiap node menyimpan transisi karakter ke node berikutnya, tautan kegagalan untuk berpindah ketika terjadi mismatch, serta daftar output keyword yang berakhir pada node tersebut. Automaton dibangun satu kali dari seluruh keyword, kemudian digunakan untuk memindai teks secara linear.

```
interface AhoCorasickNode {
  transitions: Map<string, number>;
  failureLink: number;
  outputs: string[];
}

export interface AhoCorasickAutomaton {
  nodes: AhoCorasickNode[];
}

export interface AhoCorasickSearchResult {
  matches: MatchResult[];
  comparisons: number;
  automaton: AhoCorasickAutomaton;
}
```

Gambar 4.1.2.6. Struktur AhoCorasickNode, AhoCorasickAutomaton dan AhoCorasickSearchResult

Pada Rabin-Karp, struktur hasil pencarian menyimpan hash pattern, jumlah perbandingan hash, jumlah perbandingan verifikasi karakter, total perbandingan, dan daftar match. Pemisahan hashComparisons dan searchComparisons dibuat karena Rabin-Karp memiliki dua tahap pemeriksaan, yaitu perbandingan hash dan verifikasi karakter ketika hash cocok.

```
export interface RabinKarpKeywordResult {
  keyword: string;
  patternHash: number;
  searchComparisons: number;
  hashComparisons: number;
  totalComparisons: number;
  matches: MatchResult[];
}

export interface RabinKarpSearchResult {
  matches: MatchResult[];
  keywordResults: RabinKarpKeywordResult[];
  comparisons: number;
}
```

Gambar 4.1.2.7. Struktur RabinKarpKeywordResult dan RabinKarpSearchResult

Untuk menyimpan hasil pencarian dan pengaturan extension, program menggunakan struktur data pada modul storage. StoredAlgorithmStat menyimpan statistik setiap algoritma, meliputi jumlah match, waktu eksekusi, dan jumlah perbandingan. StoredKeywordStat menyimpan jumlah kemunculan setiap keyword yang ditemukan. Kedua struktur tersebut kemudian digunakan di dalam

StoredSearchStats, yaitu struktur utama untuk menyimpan hasil scan pada suatu URL. Selain itu, JudolSettings digunakan untuk menyimpan status pengaturan extension, seperti aktif atau tidaknya deteksi, blur, Aho-Corasick, Rabin-Karp, dan OCR.

```
export interface StoredAlgorithmStat {
  matchCount: number;
  executionTimeMs: number;
  comparisons: number;
}

export interface StoredKeywordStat {
  keyword: string;
  matchCount: number;
}

export interface StoredSearchStats {
  searchedAt: number;
  url: string;
  totalKeywordsFound: number;
  totalMatches: number;
  keywordCounts: StoredKeywordStat[];
  algorithmStats: Partial<Record<MatchingAlgorithm, StoredAlgorithmStat>>;
}

export interface JudolSettings {
  enabled: boolean;
  blurEnabled: boolean;
  ahoCorasickEnabled: boolean;
  rabinKarpEnabled: boolean;
  ocrEnabled: boolean;
}
```

Gambar 4.1.2.9. Struktur StoredAlgorithmStat, StoredKeywordStat, StoredSearchStats, dan JudolSettings

Untuk fitur OCR, hasil deteksi gambar disimpan dalam OcrImageMatchResult. Struktur ini menyimpan elemen gambar, URL sumber gambar, teks hasil OCR, dan match yang ditemukan. Dengan struktur ini, program dapat menghubungkan hasil OCR dengan elemen gambar yang perlu disensor.

```
export interface OcrImageMatchResult {
  element: HTMLImageElement;
  sourceUrl: string;
  recognizedText: string;
  match: MatchResult;
}

export interface OcrImageScanResult {
  matches: OcrImageMatchResult[];
  scannedImageCount: number;
  detectedImageCount: number;
  comparisons: number;
}
```

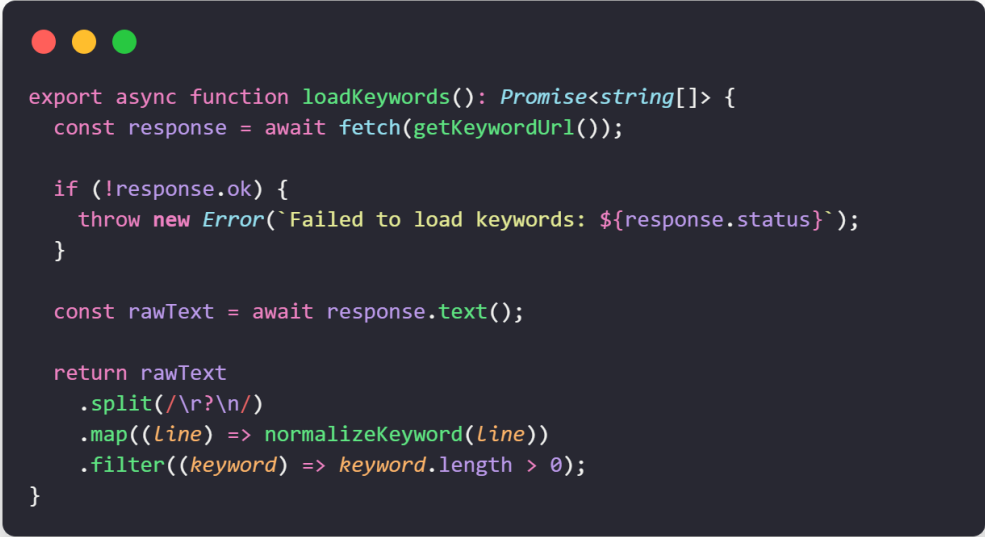

Gambar 4.1.2.9. Struktur `OcrImageMatchResult` dan `OcrImageScanResult`

Secara keseluruhan, struktur data pada program dirancang agar hasil dari berbagai algoritma tetap memiliki format yang seragam. Hal ini memudahkan integrasi antar modul, karena hasil KMP, Boyer-Moore, Regex, Weighted Levenshtein, Aho-Corasick, Rabin-Karp, dan OCR dapat diproses bersama oleh sistem highlight, tooltip, statistik popup, dan penyimpanan hasil pencarian.

4.1.3. Implementasi Program

4.1.3.1. Pemuatan keyword dan pengambilan text node

Tahap awal deteksi dilakukan dengan memuat daftar keyword dari file `keyword.txt`. File ini dibaca melalui URL extension, kemudian setiap barisnya dinormalisasi menjadi bentuk lowercase dan baris kosong diabaikan. Hasil dari tahap ini adalah array keyword yang akan digunakan sebagai pattern pada algoritma pencocokan string.



```
export async function loadKeywords(): Promise<string[]> {
  const response = await fetch(getKeywordUrl());

  if (!response.ok) {
    throw new Error(`Failed to load keywords: ${response.status}`);
  }

  const rawText = await response.text();

  return rawText
    .split(/\r?\n/)
    .map((line) => normalizeKeyword(line))
    .filter((keyword) => keyword.length > 0);
}
```

Gambar 4.1.3.1.1. Implementasi Fungsi `loadKeywords()`

Setelah keyword dimuat, program mengambil teks dari halaman web menggunakan `TreeWalker`. Text node yang berada di dalam elemen seperti `script`, `style`, `input`, dan `textarea` dilewati agar program hanya memproses konten yang relevan dan terlihat oleh pengguna.

```
export function collectTextNodes(root: ParentNode = document.body):  
TextNodeInfo[] {  
  const textNodes: TextNodeInfo[] = [];  
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT);  
  
  let currentNode = walker.nextNode();  
  
  while (currentNode) {  
    const textNode = currentNode as Text;  
    const text = textNode.nodeValue ?? "";  
  
    if (!shouldSkipTextNode(textNode) && !isBlankText(text)) {  
      textNodes.push({ node: textNode, text });  
    }  
  
    currentNode = walker.nextNode();  
  }  
  
  return textNodes;  
}
```

Gambar 4.1.3.1.2. Implementasi Fungsi collectTextNodes()

Keyword dan teks halaman kemudian dinormalisasi agar pencocokan tidak sensitif terhadap huruf besar dan kecil. Dengan demikian, kata seperti SLOT, Slot, dan slot tetap diproses sebagai bentuk yang sama.

```
export function normalizeKeyword(keyword: string): string {  
  return keyword  
    .normalize("NFKD")  
    .replace(/[\u0300-\u036f]/g, EMPTY_TEXT)  
    .replace(/\s+/g, " ")  
    .trim()  
    .toLowerCase();  
}
```

Gambar 4.1.3.1.3. Implementasi Fungsi normalizeKeyword()

Pada alur utama program, hasil pemuatan keyword dan pengambilan text node digunakan sebagai input untuk seluruh algoritma deteksi.

```
const keywords = await loadKeywords();
const textNodes = collectTextNodes();
const keyStat: KeywordStats = new Map();
const algorithmStats = createEmptyAlgorithmStats();
```

Gambar 4.1.3.1.4. Pemuatan Keyword dan Pengambilan Text Node di Alur Utama Program

Dengan tahap ini, masalah deteksi konten judi online dipetakan menjadi persoalan pencocokan antara kumpulan keyword sebagai pattern dan text node halaman sebagai text.

4.1.3.2. Implementasi KMP

Algoritma KMP digunakan untuk melakukan exact matching antara keyword dan teks halaman. Implementasi dimulai dengan membangun tabel LPS. Tabel ini menyimpan informasi prefix terpanjang yang juga merupakan suffix pada pattern, sehingga ketika terjadi mismatch algoritma tidak perlu mengulang pencocokan dari awal.

```
export function buildKmpLpsTable(pattern: string): { lpsTable: number[];
comparisons: number } {
  const lpsTable: number[] = new Array(pattern.length).fill(0);
  let prefixLength = 0;
  let suffixIndex = 1;
  let comparisons = 0;

  while (suffixIndex < pattern.length) {
    comparisons += 1;

    if (pattern[suffixIndex] === pattern[prefixLength]) {
      prefixLength += 1;
      lpsTable[suffixIndex] = prefixLength;
      suffixIndex += 1;
    } else if (prefixLength > 0) {
      prefixLength = lpsTable[prefixLength - 1];
    } else {
      lpsTable[suffixIndex] = 0;
      suffixIndex += 1;
    }
  }

  return { lpsTable, comparisons };
}
```

Gambar 4.1.3.2.1. Implementasi Fungsi buildKmpLpsTable()

Setelah tabel LPS terbentuk, proses pencarian dilakukan dengan dua indeks, yaitu textIndex untuk teks halaman dan patternIndex untuk keyword. Jika karakter cocok, kedua indeks bergerak maju. Jika seluruh keyword berhasil dicocokkan, program menyimpan informasi

hasil deteksi berupa keyword, algoritma, posisi awal dan akhir, teks yang cocok, jumlah perbandingan, serta waktu pencarian. Jika terjadi mismatch, KMP tidak mengembalikan indeks teks ke posisi awal, tetapi menggeser patternIndex menggunakan tabel LPS.

```
while (textIndex < text.length) {
  searchComparisons += 1;

  if (text[textIndex] === keyword[patternIndex]) {
    textIndex += 1;
    patternIndex += 1;

    if (patternIndex === keyword.length) {
      const startIndex = textIndex - keyword.length;
      const endIndex = textIndex;

      const endTime = performance.now();
      const segmentDuration = endTime - startTime;

      matches.push({
        keyword,
        algorithm: "KMP",
        startIndex,
        endIndex,
        matchedText: text.slice(startIndex, endIndex),
        comparisons: preprocessingComparisons + searchComparisons,
        searchTime: segmentDuration
      });

      patternIndex = lpsTable[patternIndex - 1];
      startTime = performance.now();
    }
  } else if (patternIndex > 0) {
    patternIndex = lpsTable[patternIndex - 1];
  } else {
    textIndex += 1;
  }
}
```

Gambar 4.1.3.2.2. Implementasi Proses Pencarian String pada Algoritma KMP pada Fungsi searchKmp()

Karena keyword yang digunakan tidak hanya satu, program menyediakan fungsi searchKmpKeywords() sebagai pembungkus proses pencarian. Fungsi ini menerima teks dan daftar keyword, lalu menjalankan searchKmp() untuk setiap keyword. Seluruh hasil pencarian kemudian digabungkan ke dalam array matches, sedangkan jumlah perbandingan dari setiap pencarian dijumlahkan ke variabel comparisons. Dengan struktur ini, content script dapat memanggil satu fungsi untuk memperoleh seluruh hasil KMP pada suatu text node.

```
export function searchKmpKeywords(text: string, keywords: readonly string
[]): KmpSearchResult {
  const keywordResults: KmpKeywordResult[] = [];
  const matches: MatchResult[] = [];
  let comparisons = 0;

  for (const keyword of keywords) {
    const result = searchKmp(text, keyword);

    keywordResults.push(result);
    comparisons += result.totalComparisons;

    for (const match of result.matches) {
      matches.push(match);
    }
  }

  return {
    matches,
    keywordResults,
    comparisons
  };
}
```

Gambar 4.1.3.2.3. Implementasi Fungsi searchKmpKeywords()

4.1.3.3. Implementasi Boyer-Moore

Boyer-Moore digunakan sebagai algoritma exact string matching kedua. Berbeda dari KMP yang mencocokkan pattern dari kiri ke kanan, Boyer-Moore melakukan pencocokan dari karakter paling kanan pattern. Implementasi ini menggunakan teknik looking-glass dan character-jump melalui tabel last occurrence, yaitu tabel yang menyimpan posisi terakhir setiap karakter pada keyword. Tabel ini dipakai untuk menentukan seberapa jauh pattern dapat digeser ketika terjadi mismatch.

```
function buildLastOccurrence(pattern: string): Map<string, number>{
  const lastOccurrence = new Map<string, number>();
  for(let i = 0; i < pattern.length; i++){
    lastOccurrence.set(pattern[i], i);
  }
  return lastOccurrence;
}
```

Gambar 4.1.3.3.1. Implementasi Fungsi buildLastOccurrence()

Setelah tabel last occurrence dibuat, fungsi searchBoyerMoore() melakukan pencarian dengan memulai perbandingan dari indeks terakhir keyword. Jika karakter cocok, indeks pattern bergerak mundur. Jika seluruh pattern cocok, posisi match disimpan ke dalam array matches.

Jika terjadi mismatch, program mengambil posisi terakhir karakter mismatch pada pattern, lalu menghitung pergeseran menggunakan aturan bad character.

```
while (textIndex <= m - n) {
  let patternIndex = n - 1;

  while (patternIndex >= 0) {
    searchComparisons++;

    if(keyword[patternIndex] === text[textIndex + patternIndex]) {
      patternIndex--;
    }else{
      break;
    }
  }

  // seluruh pattern cocok
  if (patternIndex < 0) {
    const startIndex = textIndex;
    const endIndex = textIndex + n;

    const endTime = performance.now();
    const segmentDuration = endTime - startTime;

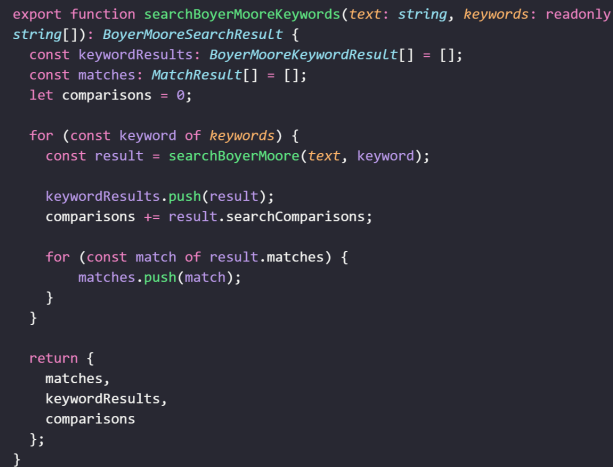
    matches.push({
      keyword,
      algorithm: "Boyer-Moore",
      startIndex,
      endIndex,
      matchedText: text.slice(startIndex, endIndex),
      comparisons: searchComparisons,
      searchTime: segmentDuration
    });

    textIndex += 1;
    startTime = performance.now();
  } else {
    const x = lastOccurrence.get(text[textIndex + patternIndex]) ?? -1;

    const shift = Math.max(1, patternIndex - x);
    textIndex += shift;
  }
}
```

Gambar 4.1.3.3.2. Implementasi Proses Pencarian String pada Algoritma Boyer Moore pada Fungsi searchBoyerMoore()

Untuk menangani seluruh daftar keyword, program menyediakan fungsi searchBoyerMooreKeywords(). Fungsi ini menjalankan searchBoyerMoore() untuk setiap keyword, menyimpan hasil tiap keyword ke keywordResults, menggabungkan semua match ke array matches, dan menjumlahkan total perbandingan karakter.



```
export function searchBoyerMooreKeywords(text: string, keywords: readonly
string[]): BoyerMooreSearchResult {
  const keywordResults: BoyerMooreKeywordResult[] = [];
  const matches: MatchResult[] = [];
  let comparisons = 0;

  for (const keyword of keywords) {
    const result = searchBoyerMoore(text, keyword);

    keywordResults.push(result);
    comparisons += result.searchComparisons;

    for (const match of result.matches) {
      matches.push(match);
    }
  }

  return {
    matches,
    keywordResults,
    comparisons
  };
}
```

Gambar 4.1.3.3.3. Implementasi Fungsi searchBoyerMooreKeywords()

4.1.3.4. Implementasi Regex

Regex digunakan untuk mendeteksi pola kata yang diikuti angka. Pola ini ditujukan untuk menangkap teks seperti GACOR99, MAXWIN88, SLOT88, atau bentuk lain yang terdiri dari huruf lalu diikuti minimal dua digit angka. Berbeda dari KMP dan Boyer-Moore yang mencocokkan teks terhadap daftar keyword, Regex mencari pola umum langsung pada teks halaman.

```

const WORD_WITH_DIGITS_PATTERN = /(?<![\p{L}\p{N}_])\p{L}[\p{L}\p{M}]*
\p{N}{2,}(?![\p{L}\p{N}_])/gu;

export function searchRegex(text: string): MatchResult[] {
  const matches: MatchResult[] = [];

  WORD_WITH_DIGITS_PATTERN.lastIndex = 0;

  let startTime = performance.now();
  let regexMatch = WORD_WITH_DIGITS_PATTERN.exec(text);

  while (regexMatch !== null) {
    const matchedText = regexMatch[0];
    const startIndex = regexMatch.index;
    const endIndex = startIndex + matchedText.length;

    const endTime = performance.now();
    const segmentDuration = endTime - startTime;

    matches.push({
      keyword: matchedText,
      algorithm: "Regex",
      startIndex,
      endIndex,
      matchedText,
      comparisons: 0,
      searchTime: segmentDuration
    });

    startTime = performance.now();
    regexMatch = WORD_WITH_DIGITS_PATTERN.exec(text);
  }

  return matches;
}

```

Gambar 4.1.3.4.1. Implementasi Fungsi searchRegex()

Pattern regex tersebut menggunakan Unicode flag u dan global flag g. Bagian `\p{L}` digunakan untuk mencocokkan huruf, sedangkan `\p{N}{2,}` digunakan untuk mencocokkan minimal dua angka setelah huruf. *Negative lookbehind* dan *negative lookahead* digunakan agar pola yang terdeteksi tidak berada di tengah kata atau token lain yang lebih panjang. Setiap hasil yang ditemukan kemudian disimpan dalam bentuk `MatchResult`, sehingga format hasilnya tetap sama dengan algoritma lain dan dapat diproses oleh sistem highlight, tooltip, dan statistik.

4.1.3.5. Implementasi Weighted Levenshtein

Weighted Levenshtein digunakan untuk mendeteksi keyword yang tidak muncul secara exact, tetapi masih memiliki kemiripan tinggi dengan keyword asli. Fitur ini ditujukan untuk menangani variasi penulisan yang sengaja dimodifikasi, misalnya penggunaan angka atau simbol sebagai pengganti huruf seperti `gac0r`, `sl0t`, atau `m4xwin`. Berbeda dari Levenshtein Distance biasa, implementasi ini memberi bobot substitusi yang berbeda untuk karakter-karakter yang mirip secara visual.


```

const INSERTION_COST = 1;
const DELETION_COST = 1;
const NORMAL_SUBSTITUTION_COST = 1;
const VISUALLY_SIMILAR_SUBSTITUTION_COST = 0.25;
const DEFAULT_SIMILARITY_THRESHOLD = 0.8;

const VISUALLY_SIMILAR_GROUPS: ReadonlyArray<ReadonlySet<string>> = [
  new Set(["a", "4", "@", "\u03b1"]),
  new Set(["o", "0"]),
  new Set(["i", "1", "l", "|", "!"]),
  new Set(["l", "1", "i", "|"]),
  new Set(["s", "5", "$"]),
  new Set(["e", "3"]),
  new Set(["g", "6", "9"]),
  new Set(["t", "7"])
];

```

Gambar 4.1.3.5.1. Pendefinisian Konstanta yang Digunakan pada Weighted Levenshtein

Nilai threshold similarity ditetapkan sebesar 0.8 agar kandidat yang diterima tetap memiliki kemiripan tinggi dengan keyword asli. Nilai ini dipilih untuk mengurangi *false positive*, yaitu kondisi ketika kata biasa yang hanya sedikit mirip dengan keyword ikut terdeteksi. Sementara itu, biaya substitusi karakter mirip visual ditetapkan sebesar 0.25, lebih kecil daripada biaya substitusi normal sebesar 1, karena pasangan seperti o dan 0, a dan 4, atau s dan 5 secara visual masih mewakili bentuk huruf yang sama dalam pola leetspeak. Daftar karakter pada VISUALLY_SIMILAR_GROUPS ditentukan berdasarkan variasi leetspeak yang umum digunakan untuk menyamarkan kata.

Untuk menentukan biaya substitusi, program menggunakan fungsi `getSubstitutionCost()`. Jika dua karakter sama, biayanya 0. Jika dua karakter berada dalam kelompok visual yang sama, biayanya 0.25. Jika tidak, substitusi dianggap sebagai perubahan karakter biasa dengan biaya 1.

```

export function getSubstitutionCost(a: string, b: string): number {
  const [leftCharacter = ""] = toComparableCharacters(a);
  const [rightCharacter = ""] = toComparableCharacters(b);

  if (leftCharacter === rightCharacter) {
    return 0;
  }

  if (areVisuallySimilar(leftCharacter, rightCharacter)) {
    return VISUALLY_SIMILAR_SUBSTITUTION_COST;
  }

  return NORMAL_SUBSTITUTION_COST;
}

```

Gambar 4.1.3.5.2. Implementasi Fungsi `getSubstitutionCost()`

Perhitungan jarak dilakukan menggunakan dynamic programming. Program hanya menyimpan dua baris tabel, yaitu `previousRow` dan `currentRow`, sehingga penggunaan memori lebih efisien. Untuk setiap pasangan karakter, program menghitung tiga kemungkinan operasi: deletion, insertion, dan substitution. Nilai terkecil dari ketiganya digunakan sebagai jarak minimum sementara.

```
function computeWeightedLevenshteinDistance(a: string, b: string):  
  WeightedDistanceResult {  
    const source = toComparableCharacters(a);  
    const target = toComparableCharacters(b);  
    let comparisons = 0;  
  
    let previousRow: number[] = new Array(target.length + 1);  
    let currentRow: number[] = new Array(target.length + 1);  
  
    for (let column = 0; column <= target.length; column += 1) {  
      previousRow[column] = column * INSERTION_COST;  
    }  
  
    for (let row = 1; row <= source.length; row += 1) {  
      currentRow[0] = row * DELETION_COST;  
  
      for (let column = 1; column <= target.length; column += 1) {  
        comparisons += 1;  
  
        const deletion = previousRow[column] + DELETION_COST;  
        const insertion = currentRow[column - 1] + INSERTION_COST;  
        const substitution = previousRow[column - 1] + getSubstitutionCost(  
          source[row - 1], target[column - 1]);  
  
        currentRow[column] = Math.min(deletion, insertion, substitution);  
      }  
  
      const finishedRow = previousRow;  
      previousRow = currentRow;  
      currentRow = finishedRow;  
    }  
  
    return {  
      distance: previousRow[target.length],  
      comparisons  
    };  
  }  
}
```

Gambar 4.1.3.5.3. Implementasi Fungsi `computeWeightedLevenshteinDistance()`

Setelah jarak diperoleh, nilai tersebut diubah menjadi similarity score. Semakin kecil jarak terhadap panjang maksimum kedua string, semakin tinggi nilai similarity. Kandidat dianggap cocok apabila similarity mencapai threshold yang telah ditentukan.

```
export function calculateSimilarity(a: string, b: string): number {
  const maxLength = Math.max(toComparableCharacters(a).length,
    toComparableCharacters(b).length);
  const { distance } = computeWeightedLevenshteinDistance(a, b);

  return calculateSimilarityFromDistance(distance, maxLength);
}

export function isFuzzyMatch(
  keyword: string,
  candidate: string,
  threshold: number = DEFAULT_SIMILARITY_THRESHOLD
): boolean {
  if (keyword.length === 0 || candidate.length === 0) {
    return false;
  }

  return calculateSimilarity(keyword, candidate) >= threshold;
}
```

Gambar 4.1.3.5.3. Implementasi Fungsi `calculateSimilarity()` dan `isFuzzyMatch()`

Pada tahap pencarian, teks terlebih dahulu dipecah menjadi token. Setiap token kandidat kemudian dibandingkan dengan keyword. Program juga melakukan filter panjang kandidat melalui `hasNearbyLength()` agar perhitungan distance tidak dilakukan pada kandidat yang terlalu jauh panjangnya dari keyword. Jika kandidat lolos filter panjang dan similarity-nya memenuhi threshold, hasil tersebut disimpan sebagai match dengan algoritma "Weighted Levenshtein".

```

for (const keyword of keywords) {
  const keywordTokens = tokenizeText(keyword);
  const tokenCount = keywordTokens.length;

  if (tokenCount === 0 || tokenCount > textTokens.length) {
    continue;
  }

  const comparableKeyword = joinTokenTexts(keywordTokens, 0, tokenCount);

  let startTime = performance.now();

  for (let tokenIndex = 0; tokenIndex <= textTokens.length - tokenCount; tokenIndex += 1) {
    const comparableCandidate = joinTokenTexts(textTokens, tokenIndex, tokenCount);

    if (!hasNearbyLength(comparableKeyword, comparableCandidate)) {
      continue;
    }

    const { distance, comparisons: distanceComparisons } = computeWeightedLevenshteinDistance(
      comparableKeyword,
      comparableCandidate
    );
    comparisons += distanceComparisons;

    const maxLength = Math.max(
      toComparableCharacters(comparableKeyword).length,
      toComparableCharacters(comparableCandidate).length
    );
    const similarity = calculateSimilarityFromDistance(distance, maxLength);

    if (similarity >= threshold) {
      const startIndex = textTokens[tokenIndex].startIndex;
      const endIndex = textTokens[tokenIndex + tokenCount - 1].endIndex;

      const endTime = performance.now();
      const segmentDuration = endTime - startTime;

      matches.push({
        keyword,
        algorithm: "Weighted Levenshtein",
        startIndex,
        endIndex,
        matchedText: text.slice(startIndex, endIndex),
        comparisons: distanceComparisons,
        searchTime: segmentDuration
      });
    }

    startTime = performance.now();
  }
}

```

Gambar 4.1.3.5.3. Implementasi Bagian Utama searchWeightedLevenshteinKeywords()

4.1.3.6. Implementasi Aho-Corasick

Aho-Corasick digunakan sebagai algoritma exact string matching tambahan yang bersifat opsional dan dapat diaktifkan pengguna melalui toggle pada popup. Berbeda dari KMP dan Boyer-Moore yang memproses satu keyword dalam satu waktu, Aho-Corasick mampu mencari seluruh keyword secara serentak hanya dengan satu kali pemindaian teks. Hal ini dicapai melalui pembangunan sebuah finite automaton berbasis trie yang dilengkapi failure link pada fase prapemrosesan, kemudian automaton tersebut digunakan untuk memindai teks secara linear tanpa backtracking.

Tahap pertama implementasi adalah pembangunan struktur trie dari seluruh daftar keyword. Setiap node pada trie direpresentasikan menggunakan antarmuka `AhoCorasickNode` yang memiliki tiga atribut: `transitions` berupa `Map<string, number>` yang memetakan karakter ke indeks node berikutnya, `failureLink` berupa number yang menunjuk ke node tujuan ketika terjadi mismatch, serta `outputs` berupa `string[]` yang menyimpan daftar keyword yang berakhir tepat di node tersebut. Fungsi `addKeywordToTrie()` menelusuri trie karakter demi karakter untuk setiap keyword dan membuat node baru apabila transisi yang diperlukan belum ada.

```
function createNode(): AhoCorasickNode {
  return {
    transitions: new Map<string, number>(),
    failureLink: 0,
    outputs: []
  };
}

function addKeywordToTrie(nodes: AhoCorasickNode[], keyword: string): void {
  let currentNodeIndex = 0;

  for (let index = 0; index < keyword.length; index += 1) {
    const character = keyword[index];
    const nextNodeIndex = nodes[currentNodeIndex].transitions.get(character);

    if (nextNodeIndex === undefined) {
      const newNodeIndex = nodes.length;
      nodes.push(createNode());
      nodes[currentNodeIndex].transitions.set(character, newNodeIndex);
      currentNodeIndex = newNodeIndex;
    } else {
      currentNodeIndex = nextNodeIndex;
    }
  }

  nodes[currentNodeIndex].outputs.push(keyword);
}
```

Gambar 4.1.3.6.1. Implementasi Fungsi `createNode()` dan `addKeywordToTrie()`

Setelah trie dasar terbentuk, tahap berikutnya adalah pembangunan failure link menggunakan fungsi `buildFailureLinks()`. Proses ini dilakukan dengan Breadth-First Search (BFS) mulai dari node-node anak langsung root, yang semuanya mengarahkan `failureLink` ke root (indeks 0). Untuk setiap node yang diproses, program menelusuri transisi yang dimilikinya. Untuk setiap karakter transisi tersebut, program mengikuti failure link dari node induk secara berulang hingga menemukan node yang memiliki transisi untuk karakter itu atau kembali ke root. Node yang ditemukan menjadi `failureLink` dari node anak bersangkutan. Pada saat yang sama, `outputs` dari node yang ditunjuk `failureLink` digabungkan ke `outputs` node anak tersebut menggunakan fungsi `appendOutputs()`. Penggabungan ini krusial agar keyword pendek yang merupakan suffix dari keyword panjang tetap terdeteksi.

```

function buildFailureLinks(nodes: AhoCorasickNode[]): void {
  const queue: number[] = [];
  let queueHead = 0;

  for (const childNodeIndex of nodes[0].transitions.values()) {
    nodes[childNodeIndex].failureLink = 0;
    queue.push(childNodeIndex);
  }

  while (queueHead < queue.length) {
    const currentNodeIndex = queue[queueHead];
    queueHead += 1;

    for (const [character, childNodeIndex] of nodes[currentNodeIndex].transitions) {
      let fallbackNodeIndex = nodes[currentNodeIndex].failureLink;

      while (fallbackNodeIndex !== 0 && nodes[fallbackNodeIndex].transitions.get(character) === undefined) {
        fallbackNodeIndex = nodes[fallbackNodeIndex].failureLink;
      }

      const fallbackTransition = nodes[fallbackNodeIndex].transitions.get(character);
      nodes[childNodeIndex].failureLink = fallbackTransition === undefined ? 0 : fallbackTransition;

      appendOutputs(nodes[childNodeIndex].outputs, nodes[nodes[childNodeIndex].failureLink].outputs);
      queue.push(childNodeIndex);
    }
  }
}

```

Gambar 4.1.3.6.2. Implementasi Fungsi buildFailureLinks()

Kedua tahap tersebut kemudian digabungkan dalam fungsi buildAhoCorasickAutomaton(). Fungsi ini menerima daftar keyword, mengabaikan keyword kosong, membangun trie melalui addKeywordToTrie() untuk setiap keyword, lalu memanggil buildFailureLinks() untuk melengkapi automaton, dan mengembalikan objek AhoCorasickAutomaton yang siap digunakan.

```

export function buildAhoCorasickAutomaton(keywords: readonly string[]): AhoCorasickAutomaton {
  const nodes: AhoCorasickNode[] = [createNode()];

  for (const keyword of keywords) {
    if (keyword.length > 0) {
      addKeywordToTrie(nodes, keyword);
    }
  }

  buildFailureLinks(nodes);

  return { nodes };
}

```

Gambar 4.1.3.6.3. Implementasi Fungsi buildAhoCorasickAutomaton()

Proses pencarian dilakukan oleh fungsi runAhoCorasickSearch(). Fungsi ini membangun automaton terlebih dahulu, lalu menelusuri teks karakter demi karakter dengan mempertahankan currentNodeIndex sebagai state aktif. Untuk setiap karakter, program mencoba mengikuti

transisi yang tersedia. Apabila tidak ada transisi untuk karakter tersebut dan state saat ini bukan root, program mengikuti failureLink secara berulang hingga menemukan node dengan transisi yang sesuai atau kembali ke root. Setelah state baru ditetapkan, program memeriksa apakah node tersebut memiliki outputs. Jika ada, setiap keyword pada outputs dicatat sebagai MatchResult dengan posisi akhir $\text{textIndex} + 1$ dan posisi awal yang dihitung mundur sesuai panjang keyword.

```
function runAhoCorasickSearch(text: string, keywords: readonly string[])
: AhoCorasickSearchResult {
  const automaton = buildAhoCorasickAutomaton(keywords);
  const matches: MatchResult[] = [];
  let currentNodeIndex = 0;
  let comparisons = 0;

  let startTime = performance.now();

  for (let textIndex = 0; textIndex < text.length; textIndex += 1) {
    const character = text[textIndex];
    comparisons += 1;

    while (currentNodeIndex !== 0 && automaton.nodes[currentNodeIndex]
      .transitions.get(character) === undefined) {
      currentNodeIndex = automaton.nodes[currentNodeIndex].failureLink;
      comparisons += 1;
    }

    const nextNodeIndex = automaton.nodes[currentNodeIndex]
      .transitions.get(character);
    currentNodeIndex = nextNodeIndex === undefined ? 0 : nextNodeIndex;

    for (const keyword of automaton.nodes[currentNodeIndex].outputs) {
      const endIndex = textIndex + 1;
      const startIndex = endIndex - keyword.length;

      const endTime = performance.now();
      const segmentDuration = endTime - startTime;

      matches.push({
        keyword,
        algorithm: "Aho-Corasick",
        startIndex,
        endIndex,
        matchedText: text.slice(startIndex, endIndex),
        comparisons,
        searchTime: segmentDuration
      });

      startTime = performance.now();
    }
  }

  return {
    matches,
    comparisons,
    automaton
  };
}
```

Gambar 4.1.3.6.4. Implementasi Proses Pencarian String pada Fungsi runAhoCorasickSearch()

Untuk memproses seluruh daftar keyword dan mengintegrasikan hasilnya ke sistem statistik, program menyediakan fungsi `searchAhoCorasickKeywords()`. Fungsi ini membangun automaton dari seluruh keyword terlebih dahulu, kemudian menjalankan `runAhoCorasickSearch()` secara terpisah untuk setiap keyword tunggal agar waktu eksekusi per keyword dapat diukur menggunakan `measureExecution()`. Jika ditemukan hasil match, statistik keyword diperbarui pada `keyStat` sesuai pola yang sama dengan fungsi-fungsi pencarian lainnya. Seluruh hasil match kemudian digabungkan ke dalam array `matches` dan total comparisons dijumlahkan.

```
export function searchAhoCorasickKeywords(text: string, keywords: readonly string[])
: AhoCorasickSearchResult {
  const automaton = buildAhoCorasickAutomaton(keywords);
  const matches: MatchResult[] = [];
  let comparisons = 0;

  for (const keyword of keywords) {
    const result = runAhoCorasickSearch(text, [keyword]);
    comparisons += result.comparisons;
    for (const match of result.matches) {
      matches.push(match);
    }
  }

  return {
    matches,
    comparisons,
    automaton
  };
}
```

Gambar 4.1.3.6.5. Implementasi Fungsi `searchAhoCorasickKeywords()`

4.1.3.7. Implementasi Rabin Karp

Rabin Karp digunakan sebagai algoritma exact string matching tambahan. Berbeda dari KMP dan Boyer-Moore yang membandingkan karakter secara langsung, Rabin Karp memanfaatkan nilai hash untuk membandingkan pattern dengan potongan teks sepanjang pattern. Jika hash pattern sama dengan hash window teks, barulah dilakukan verifikasi karakter untuk memastikan bahwa kecocokan tersebut bukan akibat collision.


```
const HASH_BASE = 256;
const HASH_MODULUS = 1_000_000_007;

function calculateInitialHash(text: string, Length: number): number {
  let hash = 0;

  for (let index = 0; index < Length; index += 1) {
    hash = (hash * HASH_BASE + text.charCodeAt(index)) % HASH_MODULUS;
  }

  return hash;
}
```

Gambar 4.1.3.7.1. Pendefinisian Konstanta yang Digunakan pada Algoritma Rabin Karp

Agar pencarian dapat berjalan efisien, Rabin Karp menggunakan rolling hash. Setelah hash window pertama dihitung, hash window berikutnya tidak dihitung ulang dari awal. Program menghapus kontribusi karakter paling kiri, menggeser hash, lalu menambahkan karakter baru di sisi kanan.

```
function calculateHighestBasePower(Length: number): number {
  let power = 1;

  for (let index = 1; index < Length; index += 1) {
    power = (power * HASH_BASE) % HASH_MODULUS;
  }

  return power;
}

function rollHash(previousHash: number, outgoingCode: number, incomingCode: number, highestBasePower: number): number {
  const withoutOutgoing = (previousHash - (outgoingCode * highestBasePower) % HASH_MODULUS + HASH_MODULUS) % HASH_MODULUS;

  return (withoutOutgoing * HASH_BASE + incomingCode) % HASH_MODULUS;
}
```

Gambar 4.1.3.7.2. Implementasi Fungsi calculateHighestBasePower() dan rollHash()

Karena dua string berbeda masih mungkin memiliki nilai hash yang sama, program tetap melakukan verifikasi karakter ketika hash pattern dan hash window bernilai sama. Fungsi ini membandingkan isi teks dan keyword secara langsung pada posisi tertentu.



```
function verifyMatchAt(text: string, keyword: string, startIndex: number):  
{ isMatch: boolean; comparisons: number } {  
  let comparisons = 0;  
  
  for (let index = 0; index < keyword.length; index += 1) {  
    comparisons += 1;  
  
    if (text[startIndex + index] !== keyword[index]) {  
      return {  
        isMatch: false,  
        comparisons  
      };  
    }  
  }  
  
  return {  
    isMatch: true,  
    comparisons  
  };  
}
```

Gambar 4.1.3.7.3. Implementasi Fungsi verifyMatchAt()

Proses utama pencarian dilakukan pada fungsi searchRabinKarp(). Program menghitung hash keyword dan hash window pertama, lalu menggeser window satu per satu sepanjang teks. Jika hash sama, program memanggil verifyMatchAt(). Jika verifikasi berhasil, hasil pencarian disimpan sebagai MatchResult.

```
const patternHash = calculateInitialHash(keyword, keyword.length);
let windowHash = calculateInitialHash(text, keyword.length);
const highestBasePower = calculateHighestBasePower(keyword.length);
let searchComparisons = 0;
let hashComparisons = 0;

let startTime = performance.now();

for (let startIndex = 0; startIndex <= text.length - keyword.length;
    startIndex += 1) {
    hashComparisons += 1;

    if (windowHash === patternHash) {
        const verification = verifyMatchAt(text, keyword, startIndex);
        searchComparisons += verification.comparisons;

        if (verification.isMatch) {
            const endIndex = startIndex + keyword.length;

            const endTime = performance.now();
            const segmentDuration = endTime - startTime;

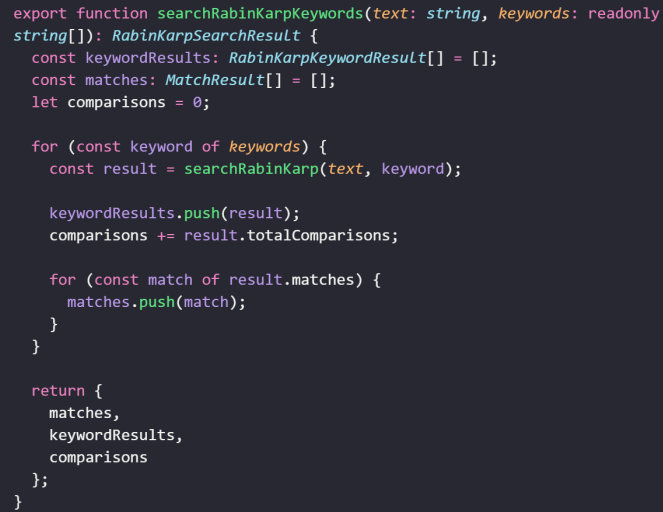
            matches.push({
                keyword,
                algorithm: "Rabin-Karp",
                startIndex,
                endIndex,
                matchedText: text.slice(startIndex, endIndex),
                comparisons: hashComparisons + searchComparisons,
                searchTime: segmentDuration
            });

            startTime = performance.now();
        }
    }

    if (startIndex < text.length - keyword.length) {
        windowHash = rollHash(
            windowHash,
            text.charCodeAt(startIndex),
            text.charCodeAt(startIndex + keyword.length),
            highestBasePower
        );
    }
}
```

Gambar 4.1.3.7.4. Implementasi Bagian Utama searchRabinKarp()

Untuk memproses seluruh daftar keyword, program menyediakan fungsi searchRabinKarpKeywords(). Fungsi ini menjalankan Rabin-Karp untuk setiap keyword, menggabungkan seluruh hasil match, dan menjumlahkan total perbandingan dari proses hash maupun verifikasi karakter.



```
export function searchRabinKarpKeywords(text: string, keywords: readonly
string[]): RabinKarpSearchResult {
  const keywordResults: RabinKarpKeywordResult[] = [];
  const matches: MatchResult[] = [];
  let comparisons = 0;

  for (const keyword of keywords) {
    const result = searchRabinKarp(text, keyword);

    keywordResults.push(result);
    comparisons += result.totalComparisons;

    for (const match of result.matches) {
      matches.push(match);
    }
  }

  return {
    matches,
    keywordResults,
    comparisons
  };
}
```

Gambar 4.1.3.7.5. Implementasi searchRabinKarpKeywords()

4.1.3.8. Highlight, tooltip, dan popup

```
1 export function highlightMatches(matches: DomMatchResult[], blurEnabled = false): Map<HTMLElement, MatchResult[]> {
2   const wrapperToMatchesMap = new Map<HTMLElement, MatchResult[]>();
3   if (matches.length === 0) return wrapperToMatchesMap;
4
5   applyContainerCensor(matches, blurEnabled);
6
7   const nodeMatchesMap = new Map<Text, DomMatchResult[]>();
8   for (const match of matches) {
9     if (!match.node) continue;
10    if (!nodeMatchesMap.has(match.node)) {
11      nodeMatchesMap.set(match.node, []);
12    }
13    nodeMatchesMap.get(match.node)!.push(match);
14  }
15
16  for (const [textNode, nodeMatches] of nodeMatchesMap.entries()) {
17    const parent = textNode.parentNode;
18    if (!parent) continue;
19
20    const fullText = typeof textNode.data === "string" ? textNode.data : "";
21    if (fullText.length === 0) continue;
22
23    const boundaries = new Set<number>();
24    boundaries.add(0);
25    boundaries.add(fullText.length);
26
27    for (const m of nodeMatches) {
28      boundaries.add(m.match.startIndex);
29      boundaries.add(m.match.endIndex);
30    }
31
32    const sortedBoundaries = Array.from(boundaries).sort((a, b) => a - b);
33
34    // Wrapper textNode awal
35    const wrapperSpan = document.createElement("span");
36    wrapperSpan.className = WRAPPER_CLASS;
37    wrapperSpan.style.display = "inline";
38
39    // Slicing
40    for (let i = 0; i < sortedBoundaries.length - 1; i++) {
41      const start = sortedBoundaries[i];
42      const end = sortedBoundaries[i + 1];
43      const sliceText = fullText.substring(start, end);
44
45      const overlappingMatches = nodeMatches.filter(
46        m => m.match.startIndex <= start && m.match.endIndex >= end
47      );
48
49      if (overlappingMatches.length === 0) {
50        wrapperSpan.appendChild(document.createTextNode(sliceText));
51      } else {
52        const markElement = document.createElement("mark");
53        markElement.className = HIGHLIGHT_CLASS;
54        markElement.dataset.judolHighlight = "true";
55        markElement.textContent = sliceText;
56
57        wrapperSpan.appendChild(markElement);
58      }
59    }
60
61    parent.replaceChild(wrapperSpan, textNode);
62
63    const pureMatches = nodeMatches.map(m => m.match);
64    wrapperToMatchesMap.set(wrapperSpan, pureMatches);
65  }
66
67  return wrapperToMatchesMap;
68 }
```

Gambar 4.1.3.8.1. Implementasi highlightMatches()

Fungsi ini digunakan untuk memanipulasi struktur DOM halaman web, lalu menandai kata kunci yang terdeteksi dengan cara mengelompokkan hasil pencarian berdasarkan node teks asalnya, memotong-motong (slicing) string teks asli berdasarkan batasan indeks awal dan akhir temuan, menyisipkan elemen `<mark>` khusus pada potongan keyword.

```

1 function generateTooltipContent(matches: MatchResult[]): string {
2   if (matches.length === 0) return "";
3
4
5   const keywordCounts = new Map<string, number>();
6   for (const m of matches) {
7     keywordCounts.set(m.keyword, (keywordCounts.get(m.keyword) ?? 0) + 1);
8   }
9
10  const uniqueKeywords = Array.from(keywordCounts.entries()).sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]));
11  const uniqueAlgorithms = Array.from(new Set(matches.map((m) => m.algorithm).filter(Boolean)));
12
13  const totalExecutionTimeMs = matches.reduce((sum, m) => sum + (m.searchTime || 0), 0);
14  const totalMatches = matches.length;
15
16  const keywordBadges = uniqueKeywords
17    .map(([[kw, count]]) => `<code class="judol-tooltip-code">${kw}</code> <small>(${count})x</small>`)
18    .join(", ");
19
20  const characterImgUrl = chrome.runtime.getURL("images/megumi.png");
21
22  return `
23    
24
25    <div class="judol-tooltip-section">
26      <span class="judol-tooltip-title">
27        Terdeteksi Konten Judol
28      </span>
29      <br/><br/>
30
31      <strong>Algoritma:</strong> ${uniqueAlgorithms.join(", ")}<br/>
32      <strong>Keywords:</strong> ${keywordBadges}<br/>
33
34      <hr style="border:0; border-top:1px solid rgba(255,255,255,0.1); margin:10px 0;" />
35
36      <strong>Total Kemunculan:</strong> ${totalMatches} match<br/>
37      <strong>Waktu Eksekusi:</strong> ${totalExecutionTimeMs.toFixed(3)} ms<br/>
38    </div>
39  `;
40 }

```

Gambar 4.1.3.8.2. Implementasi generateTooltipContent()

Fungsi ini bertujuan untuk menyusun konten informasi di dalam kotak tooltip secara dinamis dengan menggabungkan data dari array MatchResult, menghitung frekuensi kemunculan dari setiap kata kunci unik yang terdeteksi, mendata algoritma apa saja yang berhasil menemukannya, serta mengakumulasi total waktu eksekusi pencarian.

```
1 export function bindTooltip(element: HTMLElement, matches: MatchResult[]): void {
2   const tooltip = initTooltip();
3
4   if (elementListenersMap.has(element)) {
5     clearTooltip(element);
6   }
7
8   const mouseEnterHandler = (e: MouseEvent) => {
9     tooltip.innerHTML = generateTooltipContent(matches);
10    tooltip.hidden = false;
11    updatePosition(e);
12  };
13
14  const mouseMoveHandler = (e: MouseEvent) => {
15    updatePosition(e);
16  };
17
18  const mouseLeaveHandler = () => {
19    tooltip.hidden = true;
20  };
21
22  element.addEventListener("mouseenter", mouseEnterHandler);
23  element.addEventListener("mousemove", mouseMoveHandler);
24  element.addEventListener("mouseleave", mouseLeaveHandler);
25
26  elementListenersMap.set(element, {
27    mouseenter: mouseEnterHandler,
28    mousemove: mouseMoveHandler,
29    mouseleave: mouseLeaveHandler
30  });
31 }
```

Gambar 4.1.3.8.3. Implementasi bindTooltip()

Fungsi ini berfungsi untuk mengikat dan menangani interaksi kursor pengguna pada elemen wrapper hasil deteksi dengan cara memasang tiga event listener utama, yaitu mouseenter untuk mengisi konten HTML sekaligus memunculkan tooltip, mousemove untuk memperbarui koordinat posisi kotak tooltip secara dinamis mengikuti pergerakan kursor mouse, dan mouseleave untuk menyembunyikan kembali kotak tooltip tersebut saat kursor bergerak keluar dari area teks.

```

1  async function initializePopup(): Promise<void> {
2    const [settings, allStats, tabs] = await Promise.all([
3      getSettings(), getSearchStats(), chrome.tabs.query({ active: true, currentWindow: true }));
4
5    const activeTab = tabs[0];
6    const activeUrl = activeTab?.url ?? "";
7    const stats = allStats[activeUrl] ?? null;
8
9    renderSettings(settings);
10   renderStats(stats);
11   bindSettingsControls();
12   bindRescanControl();
13
14   chrome.storage.onChanged.addListener(async (changes, areaName) => {
15     if (areaName !== "local") {
16       return;
17     }
18
19     if (changes[SETTINGS_STORAGE_KEY]?.newValue) {
20       renderSettings(changes[SETTINGS_STORAGE_KEY].newValue as JudolSettings);
21     }
22
23     if (changes[SEARCH_STATS_STORAGE_KEY]) {
24       const [currentActiveTab] = await chrome.tabs.query({ active: true, currentWindow: true });
25       const currentActiveUrl = currentActiveTab?.url ?? "";
26
27       const newAllStats = changes[SEARCH_STATS_STORAGE_KEY].newValue as Record<string, StoredSearchStats> | undefined;
28
29       renderStats(newAllStats?.[currentActiveUrl] ?? null);
30     }
31   });
32 }
33

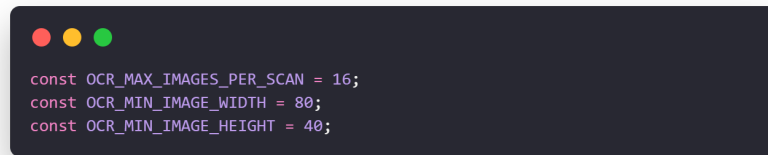
```

Gambar 4.1.3.8.3. Implementasi initializePopup()

Fungsi ini berfungsi untuk menginisialisasi tampilan jendela popup extension secara real-time saat pertama kali dibuka dengan cara mengambil data konfigurasi pengaturan dan statistik pencarian dari storage berdasarkan URL tab yang sedang aktif, merendernya ke elemen UI popup.

4.1.3.9. Implementasi OCR

OCR digunakan untuk mendeteksi konten judi online yang muncul di dalam gambar. Fitur ini diperlukan karena tidak semua teks pada halaman web berada dalam bentuk text node DOM. Pada beberapa kasus, terutama iklan atau hasil pencarian gambar, teks promosi dapat disisipkan langsung ke dalam file gambar sehingga tidak dapat dideteksi oleh KMP, Boyer-Moore, Regex, maupun Weighted Levenshtein secara langsung. Untuk menangani hal tersebut, program menggunakan Tesseract.js untuk mengekstraksi teks dari gambar, lalu teks hasil OCR diperiksa kembali menggunakan algoritma pencarian yang sudah tersedia.



```
const OCR_MAX_IMAGES_PER_SCAN = 16;  
const OCR_MIN_IMAGE_WIDTH = 80;  
const OCR_MIN_IMAGE_HEIGHT = 40;
```

Gambar 4.1.3.9.1. Pendefinisian Konstanta yang Digunakan pada OCR

Konstanta `OCR_MAX_IMAGES_PER_SCAN` digunakan untuk membatasi jumlah gambar yang diproses dalam satu kali pemindaian. Nilai ini ditetapkan sebesar 16 agar proses OCR tidak terlalu berat, karena OCR membutuhkan waktu eksekusi yang jauh lebih besar dibandingkan pencarian teks biasa. Konstanta `OCR_MIN_IMAGE_WIDTH` dan `OCR_MIN_IMAGE_HEIGHT` digunakan sebagai batas minimum ukuran gambar, yaitu 80 x 40 piksel. Batas ini bertujuan agar program tidak memproses gambar kecil seperti ikon, favicon, tombol, atau elemen dekoratif yang umumnya tidak mengandung teks penting.

Gambar yang akan dipindai terlebih dahulu difilter berdasarkan ukuran dan kondisi pemuatan gambar. Hanya gambar yang sudah selesai dimuat dan memiliki ukuran tampilan cukup besar yang akan diproses oleh OCR.

```
function isScannableImage(image: HTMLImageElement): boolean {
  const sourceUrl = getImageUrl(image);

  if (!sourceUrl || sourceUrl.startsWith("chrome-extension://")) {
    return false;
  }

  if (!image.complete || image.naturalWidth < OCR_MIN_IMAGE_WIDTH || image
.naturalHeight < OCR_MIN_IMAGE_HEIGHT) {
    return false;
  }

  const rect = image.getBoundingClientRect();

  return rect.width >= OCR_MIN_IMAGE_WIDTH && rect.height >=
OCR_MIN_IMAGE_HEIGHT;
}

function collectScannableImages(): HTMLImageElement[] {
  return Array.from(document.images)
    .filter(isScannableImage)
    .sort((left, right) => {
      const leftRect = left.getBoundingClientRect();
      const rightRect = right.getBoundingClientRect();

      return rightRect.width * rightRect.height - leftRect.width *
leftRect.height;
    })
    .slice(0, OCR_MAX_IMAGES_PER_SCAN);
}
```

Gambar 4.1.3.9.2. Implementasi Fungsi `isScannableImage()` dan `collectScannableImages()`

Setelah gambar dipilih, sistem menyiapkan input untuk Tesseract.js. Pada beberapa halaman, gambar berasal dari domain berbeda sehingga akses langsung dari content script dapat terhalang oleh kebijakan CORS. Untuk mengatasi hal tersebut, OCR menggunakan background service worker melalui mekanisme message passing. Content script mengirim URL gambar ke background, kemudian background mengambil gambar dan mengembalikannya dalam bentuk `dataUrl`.

```
async function fetchImageDataUrlViaBackground(url: string): Promise<string> {  
  const response = await chrome.runtime.sendMessage({  
    type: OCR_FETCH_IMAGE_MESSAGE_TYPE,  
    url  
  }) as OcrFetchImageResponse | undefined;  
  
  if (!response?.ok || !response.dataUrl) {  
    throw new Error(response?.error ?? "Background image fetch failed");  
  }  
  
  return response.dataUrl;  
}
```

Gambar 4.1.3.9.3. Implementasi Fungsi fetchImageDataUrlViaBackground()

Gambar yang sudah dapat diakses kemudian dimuat ulang sebagai elemen gambar sementara, digambar ke canvas, lalu dikonversi menjadi Blob PNG. Format Blob ini digunakan sebagai input OCR agar Tesseract.js membaca data gambar yang sudah siap diproses.

```
async function convertImageToPngBlob(image: HTMLImageElement): Promise<  
  Blob> {  
  const canvas = document.createElement("canvas");  
  const width = image.naturalWidth || image.width;  
  const height = image.naturalHeight || image.height;  
  const context = canvas.getContext("2d");  
  
  if (!context || width === 0 || height === 0) {  
    throw new Error("Failed to prepare fetched OCR image");  
  }  
  
  canvas.width = width;  
  canvas.height = height;  
  context.drawImage(image, 0, 0);  
  
  return canvasToPngBlob(canvas);  
}
```

Gambar 4.1.3.9.4. Implementasi Fungsi convertImageToPngBlob()

Proses pembacaan teks dilakukan pada fungsi recognizeImageText. Fungsi ini memuat Tesseract.js secara dinamis, membuat input gambar dalam bentuk Blob, lalu menjalankan recognize. Hasil teks OCR juga disimpan dalam cache berdasarkan URL dan ukuran asli gambar agar gambar yang sama tidak perlu diproses ulang pada scan berikutnya.

```
function loadTesseract(): Promise<TesseractModule> {  
  if (!tesseractModulePromise) {  
    tesseractModulePromise = import("tesseract.js") as Promise<  
TesseractModule>;  
  }  
  
  return tesseractModulePromise;  
}  
  
async function recognizeImageText(image: HTMLImageElement): Promise<string>  
> {  
  const cacheKey = getImageCacheKey(image);  
  const cachedText = recognizedTextCache.get(cacheKey);  
  
  if (cachedText !== undefined) {  
    return cachedText;  
  }  
  
  const tesseract = await loadTesseract();  
  const result = await tesseract.recognize(image, OCR_LANGUAGE);  
  const text = result.data.text.trim();  
  
  recognizedTextCache.set(cacheKey, text);  
  
  return text;  
}
```

Gambar 4.1.3.9.5. Implementasi Fungsi loadTesseract() dan recognizeImageText()

Teks hasil OCR kemudian diperiksa menggunakan beberapa metode pencarian. KMP digunakan untuk exact matching terhadap daftar keyword, Regex digunakan untuk mendeteksi pola kata yang diikuti angka, dan Weighted Levenshtein digunakan untuk mendeteksi variasi yang mirip secara visual.

```
function detectOcrTextMatches(  
  recognizedText: string,  
  keywords: readonly string[]  
) { matches: MatchResult[]; comparisons: number } {  
  const normalizedText = normalizeTextForExactSearch(recognizedText);  
  const tempKeyStat: KeywordStats = new Map();  
  const exactResult = searchKmpKeywords(normalizedText, keywords,  
    tempKeyStat);  
  const regexMatches = searchRegex(recognizedText, tempKeyStat);  
  const fuzzyResult = searchWeightedLevenshteinKeywords(normalizedText,  
    keywords, tempKeyStat);  
  const exactRanges = exactResult.matches;  
  const fuzzyMatches = fuzzyResult.matches.filter(  
    (fuzzyMatch) => !exactRanges.some((exactMatch) => rangesOverlap(  
      fuzzyMatch, exactMatch))  
  );  
  const matches = [  
    ...exactResult.matches.map((match) => toOcrMatch(match, recognizedText)),  
    ...regexMatches.map((match) => toOcrMatch(match, recognizedText)),  
    ...fuzzyMatches.map((match) => toOcrMatch(match, recognizedText))  
  ];  
  
  return {  
    matches,  
    comparisons: exactResult.comparisons + fuzzyResult.comparisons  
  };  
}
```

Gambar 4.1.3.9.6. Implementasi Fungsi detectOcrTextMatches()

Proses utama OCR dilakukan pada scanImagesWithOcr(). Fungsi ini mengambil gambar yang lolos filter, menjalankan OCR pada setiap gambar, lalu memasukkan setiap match ke array hasil. Jika OCR gagal pada salah satu gambar, gambar tersebut dilewati dan proses tetap dilanjutkan untuk gambar berikutnya. Setelah scan selesai, statistik OCR per keyword juga diperbarui.

```
export async function scanImagesWithOcr(  
  keywords: readonly string[],  
  keyStat: KeywordStats  
) : Promise<OcrImageScanResult> {  
  const startTime = nowMs();  
  const images = collectScannableImages();  
  const matches: OcrImageMatchResult[] = [];  
  let comparisons = 0;  
  
  for (const image of images) {  
    try {  
      const recognizedText = await recognizeImageText(image);  
  
      if (recognizedText.length === 0) {  
        continue;  
      }  
  
      const result = detectOcrTextMatches(recognizedText, keywords);  
      comparisons += result.comparisons;  
  
      for (const match of result.matches) {  
        matches.push({  
          element: image,  
          sourceUrl: getImageUrl(image),  
          recognizedText,  
          match  
        });  
      }  
    } catch (error) {  
      console.warn("[Judol Detector] OCR image skipped", error);  
    }  
  }  
  
  addOcrKeywordStats(  
    keyStat,  
    matches.map((match) => match.match),  
    nowMs() - startTime  
  );  
  
  return {  
    matches,  
    scannedImageCount: images.length,  
    detectedImageCount: new Set(matches.map((match) => match.element)).  
    size,  
    comparisons  
  };  
}
```

Gambar 4.1.3.9.7. Implementasi Fungsi scanImagesWithOcr()

Jika terdapat gambar yang terdeteksi mengandung keyword atau pola mencurigakan, program memberikan class sensor pada gambar tersebut. Class judol-ocr-detected menandai gambar sebagai hasil deteksi OCR, sedangkan judol-ocr-blur memberikan efek blur. Selain itu, tooltip dipasang pada gambar agar informasi deteksi dapat ditampilkan ketika gambar di-hover.

```
export function applyOcrImageCensorship(  
  matches: readonly OcrImageMatchResult[],  
  keyStat: KeywordStats  
) : number {  
  const matchesByImage = new Map<HTMLImageElement, MatchResult[]>();  
  
  for (const { element, match } of matches) {  
    if (!matchesByImage.has(element)) {  
      matchesByImage.set(element, []);  
    }  
  
    matchesByImage.get(element)!.push(match);  
  }  
  
  for (const [image, imageMatches] of matchesByImage) {  
    image.classList.add(OCR_DETECTED_CLASS);  
    image.classList.add(OCR_BLUR_CLASS);  
    image.setAttribute(OCR_DATA_ATTR, "true");  
    bindTooltip(image, imageMatches, keyStat);  
  }  
  
  return matchesByImage.size;  
}
```

Gambar 4.1.3.9.8. Implementasi Fungsi applyOcrImageCensorship()

4.2. Tata Cara Penggunaan Extension

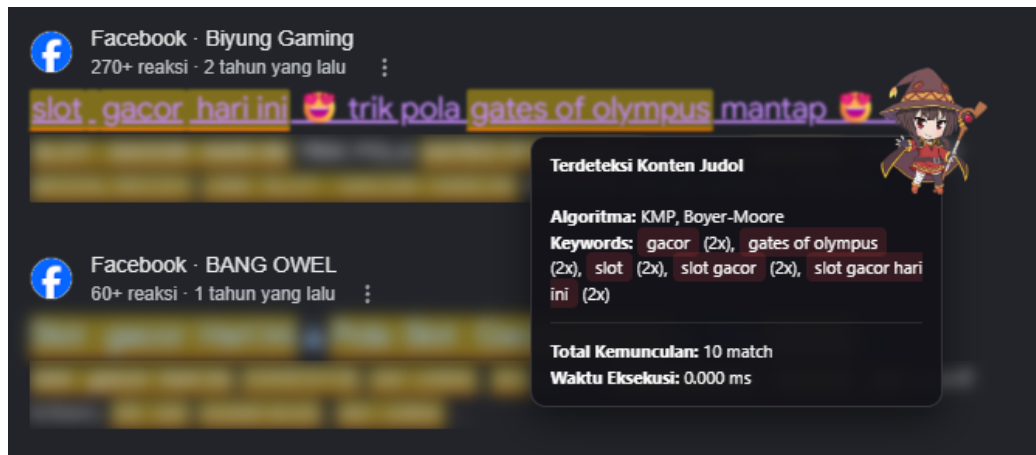
Untuk menggunakan Judol Detector, pengguna terlebih dahulu harus melakukan instalasi dengan langkah berikut: (1) pastikan Node.js dan npm telah terpasang, (2) jalankan npm install pada direktori root untuk mengunduh dependensi, (3) jalankan npm run build untuk mengompilasi TypeScript dan menghasilkan berkas di folder dist/, (4) buka Google Chrome atau browser Chromium lainnya dan navigasikan ke chrome://extensions/, (5) aktifkan Developer Mode melalui toggle di pojok kanan atas, (6) klik tombol Load unpacked, (7) pilih folder dist/ hasil build. Extension akan langsung aktif dan terdaftar di daftar ekstensi browser.

Setelah terpasang, extension berjalan otomatis saat pengguna membuka halaman web. Teks yang mengandung kata kunci judul akan diberi highlight berwarna. Untuk melihat detail deteksi, pengguna dapat mengarahkan kursor ke teks yang di-highlight; tooltip akan muncul menampilkan keyword yang terdeteksi, algoritma yang digunakan, jumlah kemunculan, dan waktu eksekusi. Untuk melihat statistik keseluruhan halaman, pengguna dapat mengklik ikon extension di toolbar browser untuk membuka popup yang menampilkan total keyword, grafik batang perbandingan keyword, dan tabel performa algoritma. Pengguna juga dapat mengaktifkan/menonaktifkan fitur Blur Teks, Aho-Corasick, dan Rabin-Karp melalui toggle di popup.

4.3. Pengujian

4.3.1. Hasil Pengujian

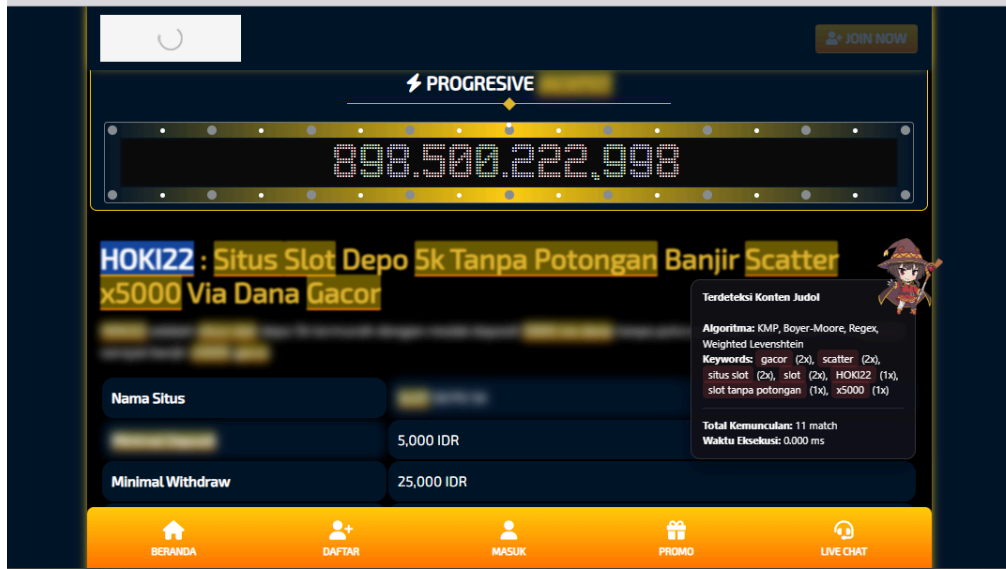
4.3.1.1. Contoh Hasil Exact Matching



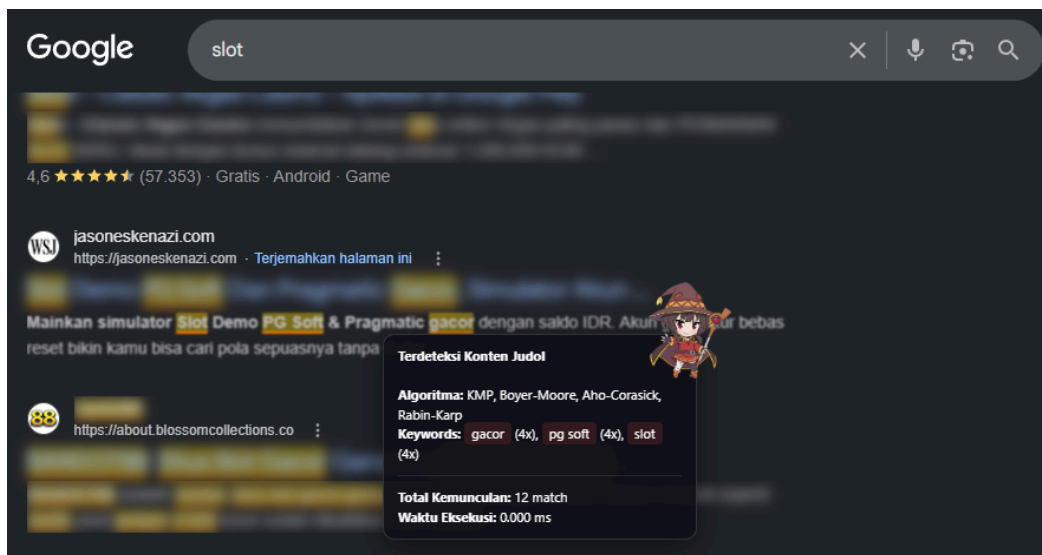
4.3.1.2. Contoh Hasil Exact Matching dan Regex



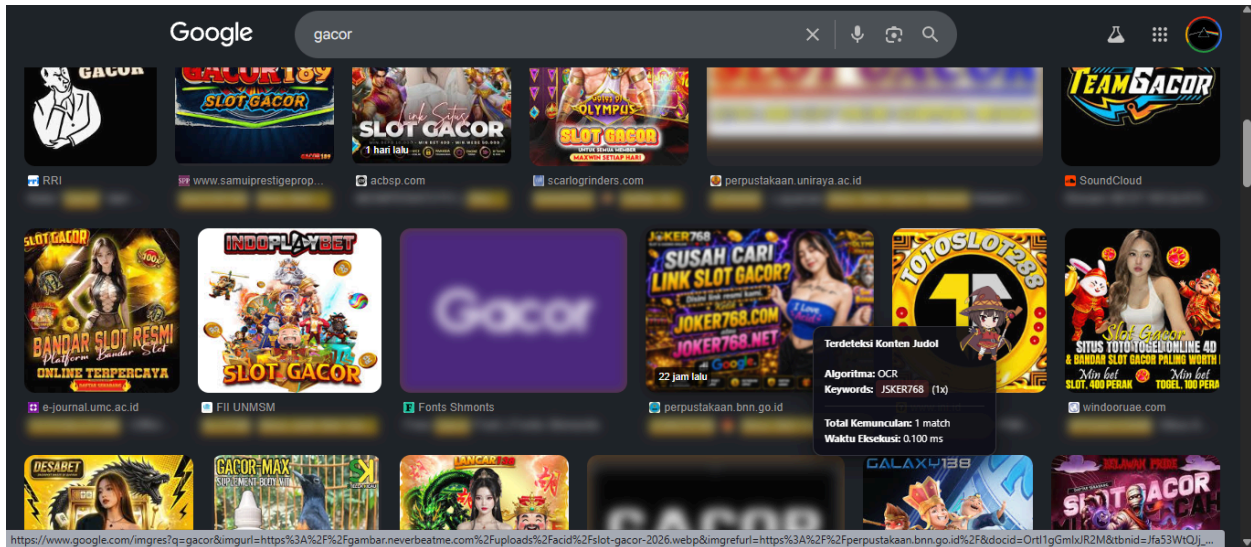
4.3.1.3. Contoh Hasil Exact Matching, Regex dan Fuzzy



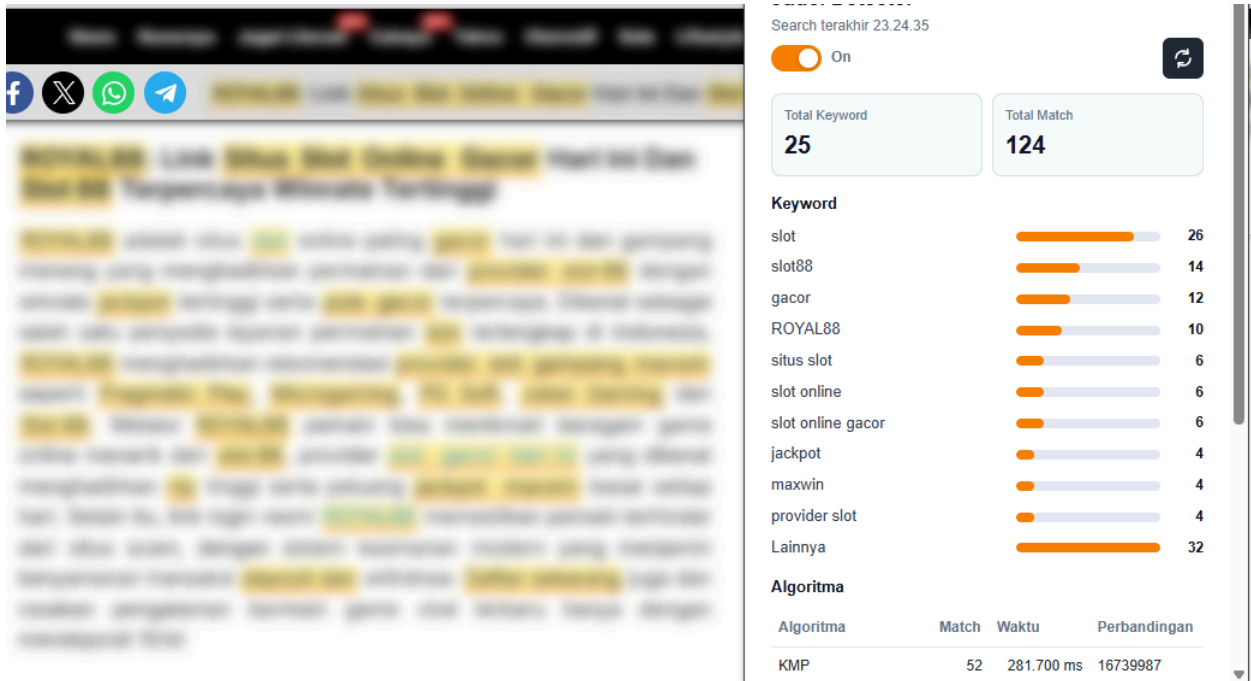
4.3.1.4. Contoh Hasil dengan Aho-Corasick dan Rabin-Karp

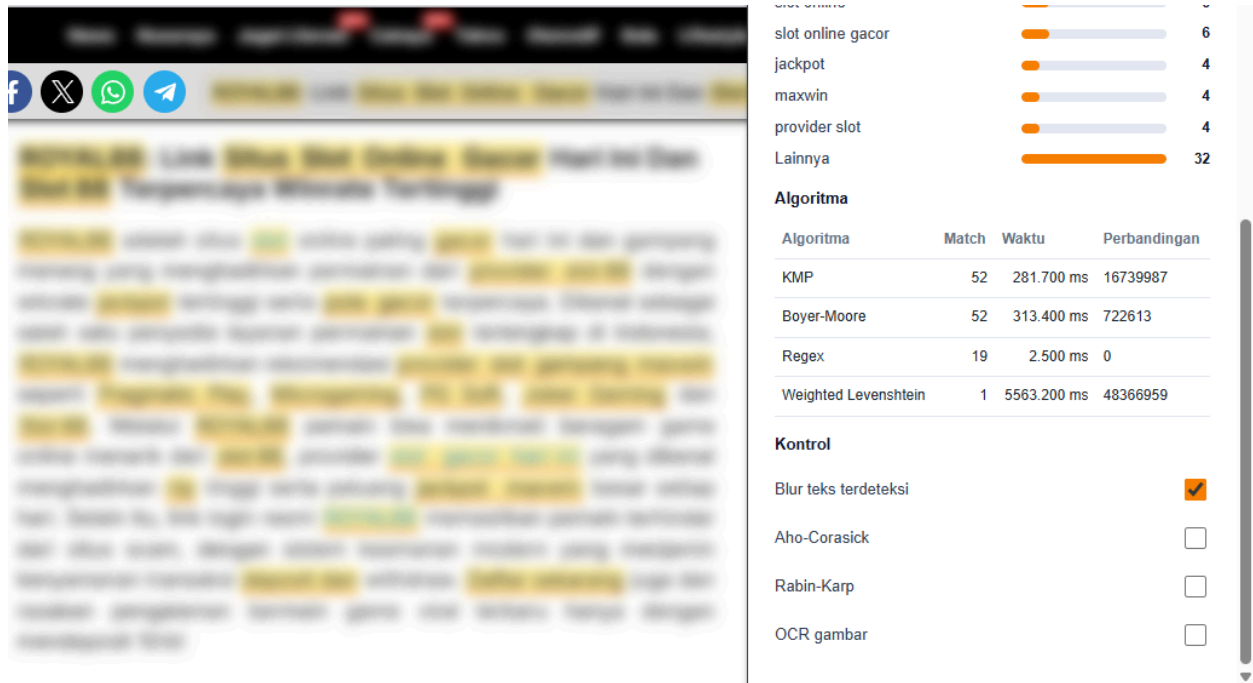


4.3.1.5. Contoh Hasil OCR



4.3.1.6. Contoh Hasil Popup





4.3.2. Analisis Hasil

Berdasarkan analisis hasil pengujian terhadap sistem Chromium browser extension untuk deteksi judi online ini, terdapat beberapa batasan teknis yang memengaruhi tingkat keberhasilan deteksi secara keseluruhan di lapangan. Kekurangan pertama terletak pada efektivitas fitur Optical Character Recognition (OCR) yang mengintegrasikan library Tesseract.js. *Library* ini dinilai tidak begitu akurat dalam mengenali teks karena tingkat presisi pembacaannya sangat bergantung pada kualitas visual. Ketika dihadapkan pada gambar dengan resolusi rendah, kontras warna yang buruk, atau tipografi teks judi yang sengaja dimanipulasi, library sering kali mengalami salah baca, sehingga kata kunci ilegal dalam gambar gagal diekstrak dan luput dari mekanisme sensor otomatis.

Kekurangan kedua bersumber dari kelemahan mekanisme DOM parser saat melakukan pengumpulan node teks menggunakan metode TreeWalker pada halaman web. Prosedur penelusuran ini melakukan traversal hingga ke unit teks terkecil yang dibungkus oleh tag-tag inline minor seperti `<em>`, `<strong>`, atau `<a>`. Akibatnya, satu kesatuan elemen yang secara semantik diekspektasikan sebagai satu blok paragraf utuh menjadi terpecah-pecah menjadi beberapa kalimat terisolasi yang akhirnya terdeteksi sebagai banyak node independen. Fragmentasi teks ini tidak hanya memicu potensi redundansi saat data dihitung untuk kebutuhan statistik atau penampilan tooltip, tetapi juga berisiko menggagalkan pencarian exact matching karena sebuah kata kunci target bisa saja tidak teridentifikasi apabila polanya terputus di tengah jalan oleh intervensi tag-tag kecil tersebut.

BAB V

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, browser extension Judol Detector berhasil memenuhi seluruh spesifikasi utama tugas besar dengan mengintegrasikan berbagai metode pattern matching, yaitu Knuth-Morris-Pratt (KMP), Boyer-Moore, Regular Expression, dan Weighted Levenshtein Distance. Sistem mampu mendeteksi kata kunci judi online pada halaman web, memberikan penandaan visual melalui fitur highlighting dan tooltip, serta menyajikan statistik performa algoritma secara real-time melalui popup extension. Selain itu, fitur bonus berupa OCR, Aho-Corasick, Rabin-Karp, dan censorship konten juga berhasil direalisasikan.

Meskipun demikian, hasil pengujian menunjukkan bahwa masih terdapat beberapa keterbatasan teknis yang memengaruhi akurasi deteksi. Fitur OCR berbasis Tesseract.js masih sensitif terhadap kualitas gambar sehingga teks pada gambar dengan resolusi rendah atau manipulasi visual tertentu belum selalu dapat dikenali dengan baik. Selain itu, mekanisme pengambilan text node menggunakan TreeWalker berpotensi menyebabkan fragmentasi teks akibat pemisahan oleh tag-tag inline, yang dapat menimbulkan redundansi data maupun mengurangi efektivitas proses pencocokan kata kunci pada beberapa kasus.

5.2. Saran

Berdasarkan hasil analisis dan keterbatasan yang ditemukan pada saat pengujian, berikut adalah beberapa saran perbaikan dan pengembangan program selanjutnya:

1. Meningkatkan kualitas pemrosesan OCR dengan menambahkan tahap preprocessing gambar (seperti peningkatan kontras, grayscale, atau noise reduction) atau mempertimbangkan penggunaan engine OCR yang lebih akurat untuk meningkatkan tingkat keberhasilan ekstraksi teks.
2. Menyempurnakan mekanisme ekstraksi teks DOM dengan menggabungkan text node yang berasal dari elemen semantik yang sama sebelum proses pencarian, sehingga fragmentasi teks akibat tag inline dapat diminimalkan dan akurasi deteksi meningkat.
3. Melakukan pengujian pada dataset dan situs web yang lebih beragam untuk mengevaluasi ketahanan sistem terhadap berbagai bentuk penyamaran kata kunci, variasi struktur DOM, serta konten berbasis gambar yang lebih kompleks.

Lampiran

Repositori Github : [Link Github](#)

Tabel Pembagian Tugas

Nama (NIM)	Pembagian Tugas
Wafiq Hibban Robbany (13524016)	Setup Extension, Algoritma KMP, OCR
Fayyaz Akmal Lauda (13524076)	Regex Matching, Fuzzy Matching, Algoritma Aho-Corasick, Algoritma Robin Karp, CSS
Syaqina Octavia Rizha (13524088)	Algoritma Boyer Moore, Tooltip, Popup, Sensor Teks

Tabel Checklist Fitur

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video		✓

10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

Pernyataan *Generative AI*

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Wafiq Hibban Robbany



Fayyaz Akmal Lauda



Syaquina Octavia Rizha

Daftar Pustaka

- **Typescript Documentation**
<https://www.typescriptlang.org/docs/>
- **Chrome Extension Documentation**
<https://developer.chrome.com/docs/extensions/>
- **Tesseract.js**
<https://tesseract.projectnaptha.com/>
- **Aho-Corasick**
<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>
- **Rabin Karp**
<https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>
- **Boyer-Moore Algorithm**
<https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>
- **Vite Documentation**
<https://vitejs.dev/guide/>
- **Text Web-Highlighter as a Chrome Extension**
<https://web-highlights.com/blog/build-a-text-web-highlighter-as-a-chrome-extension/>